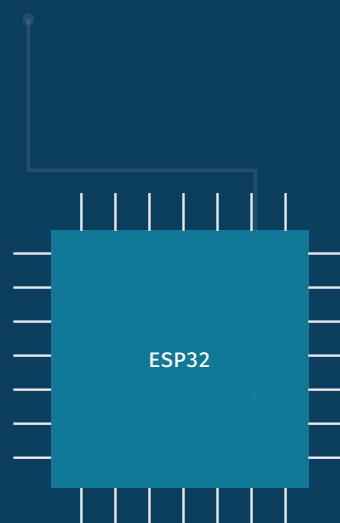


Introdução à Linguagem C para Sistemas Embarcados

Programação do ESP32 com o PlatformIO

Dos fundamentos da linguagem C ao controle de periféricos em microcontroladores: uma introdução prática à programação voltada para sistemas embarcados.



Prefácio

Esta apostila foi escrita para quem está dando os primeiros passos em programação e, ao mesmo tempo, quer entender *para onde* esses passos levam quando o assunto é **sistemas embarcados**. Em vez de tratar a linguagem C como um fim em si mesma — como costuma acontecer em cursos introdutórios genéricos —, cada conceito aqui apresentado é conduzido com um destino em mente: o controle de um microcontrolador real, o **ESP32**, programado através do ambiente **PlatformIO**.

Por que C e por que o ESP32?

A linguagem C continua sendo, décadas após sua criação, a linguagem mais próxima do hardware que ainda é acessível a iniciantes. Ela expõe conceitos que linguagens de alto nível escondem — como a organização da memória, o tamanho exato de cada tipo de dado e a manipulação direta de endereços — e é exatamente esse tipo de controle que se exige ao programar um microcontrolador, onde os recursos de memória e processamento são limitados e cada byte importa.

O ESP32 foi escolhido como plataforma de referência por ser barato, extremamente popular, bem documentado e por reunir em um único chip um processador dual-core, Wi-Fi, Bluetooth e uma quantidade generosa de periféricos (GPIOs, conversores analógico-digitais, saídas PWM, interfaces de comunicação serial). Isso permite que, com poucas linhas de código C, o estudante veja resultados físicos e imediatos: um LED que acende, um sensor que responde, um motor que gira.

Como este material está organizado

A apostila é dividida em duas partes:

- **Parte I — Fundamentos da Linguagem C:** cobre a sintaxe, os tipos de dados, as estruturas de controle, funções, arranjos, ponteiros, alocação de memória e organização de código em múltiplos arquivos. Os exemplos desta parte rodam tanto em um computador comum (compilados com `gcc`) quanto, quando fizer sentido, diretamente no ESP32.
- **Parte II — Sistemas Embarcados com o ESP32 e o PlatformIO:** apresenta o ambiente de desenvolvimento PlatformIO, a arquitetura do ESP32 e o uso de seus periféricos — entradas e saídas digitais, PWM, conversão analógico-digital, comunicação serial, interrupções e temporizadores — sempre com projetos completos e funcionais.

Ao longo dos capítulos você encontrará três tipos de caixas de destaque:

Informações complementares que aprofundam ou contextualizam o conceito tratado.

Sugestões práticas, atalhos e boas práticas de programação.

Armadilhas comuns e erros frequentes — vale a pena prestar atenção especial aqui.

O que você vai precisar

- Um computador com [Visual Studio Code](#) e a extensão **PlatformIO IDE** instalada (o Capítulo 14 explica o processo completo);
- Uma placa de desenvolvimento com o chip **ESP32** (qualquer variante — DevKit, WROOM, WROVER — e um cabo USB compatível);
- Curiosidade para errar, compilar de novo e tentar outra vez — é assim que se aprende a programar.

Bons estudos!

Sumário

Prefácio	i
I Fundamentos da Linguagem C	1
1 Primeiros Passos	3
1.1 O primeiro programa	3
1.1.1 Comentários	4
1.1.2 A diretiva <code>#include</code>	4
1.1.3 A função <code>main</code>	4
1.1.4 A instrução <code>printf</code>	5
1.1.5 O retorno e o ponto e vírgula	5
1.2 Compilando e executando	5
1.3 E no ESP32?	6
2 Sistemas Numéricos, Memória e Compilação	7
2.1 Sistema binário	7
2.2 Sistemas octal e hexadecimal	7
2.3 Memória	10
2.4 Pré-processamento	10
2.5 Compilação	12
2.6 Números pseudoaleatórios	12
3 Tipos de Variáveis e Operadores	15
3.1 Tipos primitivos	15
3.2 Tipos de largura fixa: <code>stdint.h</code>	16
3.3 Operadores	16
3.3.1 Aritméticos	17
3.3.2 Relacionais e lógicos	17
3.3.3 Bit a bit (<i>bitwise</i>)	17
3.3.4 Atribuição composta	18
3.4 Conversão de tipos (<i>casting</i>)	19
4 Entrada e Saída de Dados	21
4.1 Saída formatada: <code>printf</code>	21
4.2 Entrada formatada: <code>scanf</code>	22

4.3	Caracteres e strings	22
4.4	Sequências de escape	23
4.5	Manipulação de arquivos	23
4.6	Saída como ferramenta de depuração	24
5	Estruturas de Decisão	27
5.1	<code>if / else</code>	27
5.2	Operador ternário	27
5.3	<code>switch</code>	28
6	Laços de Repetição e Desvios	31
6.1	<code>for</code>	31
6.2	<code>while</code>	31
6.3	<code>do-while</code>	32
6.4	<code>break</code> e <code>continue</code>	33
7	Funções e Protótipos	35
7.1	Declarando e chamando funções	35
7.2	Passagem de parâmetros por valor	35
7.3	Protótipos de funções	36
7.4	Escopo de variáveis	37
7.4.1	Variáveis locais <code>static</code>	37
8	Arranjos: Vetores	39
8.1	Declarando e percorrendo um vetor	39
8.2	Vetores e strings	40
8.3	Passando arranjos para funções	40
9	Ponteiros e Alocação Dinâmica de Memória	43
9.1	O que é um ponteiro	43
9.2	Ponteiros como parâmetros de função	44
9.3	Ponteiros e vetores	44
9.4	Alocação dinâmica de memória	45
10	Structs e Enumerações	47
10.1	Registros e <code>structs</code>	47
10.1.1	Structs e ponteiros	47
10.2	Enumerações (<code>enum</code>)	48
11	Modularização: Bibliotecas e Múltiplos Arquivos	51
11.1	Bibliotecas de software	51
11.2	Dividindo um programa em múltiplos arquivos	52

II	Sistemas Embarcados com o ESP32 e o PlatformIO	55
12	Introdução aos Sistemas Embarcados	57
12.1	O que é um sistema embarcado	57
12.2	O microcontrolador ESP32	57
12.3	Como um firmware é diferente de um programa comum	58
13	Microcontroladores: Histórico e Arquitetura	61
13.1	Um breve histórico	61
13.2	A unidade de processamento	62
13.3	Arquitetura de laço único (<i>Super Loop</i>)	63
13.4	Watchdog Timer	64
14	PlatformIO: Ambiente de Desenvolvimento	67
14.1	O que é o PlatformIO	67
14.2	Instalação	68
14.3	Criando um novo projeto	68
14.4	Anatomia de um projeto PlatformIO	68
14.4.1	O arquivo <code>platformio.ini</code>	69
14.5	A barra de tarefas do PlatformIO	70
15	Anatomia de um Projeto Arduino	71
15.1	As funções <code>setup</code> e <code>loop</code>	71
15.2	Um primeiro exemplo completo	72
15.3	Compilando, carregando e monitorando	72
15.4	De volta aos fundamentos: o que você já sabe aplicar	73
16	Entradas e Saídas Digitais (GPIO)	75
16.1	Configurando um pino como saída: o LED	75
16.2	Configurando um pino como entrada: o botão	76
16.3	Controlando vários pinos com um vetor	77
17	Leitura de Sensores Analógicos (ADC)	79
17.1	Como um ADC funciona	79
17.2	Lendo um sensor analógico	80
17.3	De leitura bruta a informação: um sensor de luminosidade	80
17.4	Suavizando leituras com um vetor: média móvel	81
18	Comunicação Serial e Depuração	83
18.1	UART e o monitor serial	83
18.1.1	As variantes de <code>Serial.print</code>	84
18.2	Saída como ferramenta de depuração, agora no firmware	84
18.3	Log por nível de severidade	84
19	Interrupções e Temporização	87

19.1	O que é uma interrupção	87
19.2	Interrupções de GPIO com <code>attachInterrupt</code>	87
19.3	Temporização sem bloquear o laço: a técnica de <code>millis()</code>	88
20	Projeto Integrador: Estação de Monitoramento IoT	91
20.1	Especificação do projeto	91
20.1.1	Materiais	91
20.1.2	Ligações sugeridas	91
20.2	Projetando a estrutura do estado	91
20.3	Firmware completo	92
20.4	Como o projeto se encaixa	94
20.5	Para continuar praticando	94
A	Tabela ASCII	97
B	Referência Rápida	99
B.1	Precedência de operadores	99
B.2	Especificadores de formato (<code>printf/scanf</code>)	100
B.3	Tipos de largura fixa (<code>stdint.h</code>)	100
B.4	Funções essenciais do framework Arduino usadas nesta apostila	100

Parte I

Fundamentos da Linguagem C

CAPÍTULO 1

Primeiros Passos

C é uma linguagem de programação criada no início dos anos 1970 por Dennis Ritchie, nos laboratórios da Bell, originalmente para reescrever o sistema operacional Unix. Mais de cinquenta anos depois, ela continua sendo uma das linguagens mais usadas do mundo — e isso não é acidente: C oferece um equilíbrio raro entre abstração suficiente para ser produtiva e proximidade suficiente do hardware para ser eficiente. É exatamente por esse motivo que praticamente todo sistema operacional, driver de dispositivo e firmware de microcontrolador — incluindo o do ESP32 — tem C em algum lugar de sua base.

A linguagem C é **compilada**: o código-fonte que escrevemos (extensão `.c`) é traduzido por um programa chamado **compilador** para linguagem de máquina antes de ser executado. Isso é diferente de linguagens **interpretadas**, como Python ou JavaScript, cujo código é lido e executado linha a linha por um interpretador em tempo de execução. Java fica no meio do caminho: compila para um bytecode intermediário, interpretado depois pela JVM. Essa distinção será retomada no Capítulo 2, quando falarmos sobre o processo de compilação em detalhe.

1.1 O primeiro programa

Por tradição — que remonta ao próprio livro de Kernighan e Ritchie sobre a linguagem —, o primeiro programa que qualquer pessoa escreve em C é o *"Hello, World!"*: um programa que apenas exibe essa mensagem na tela.

hello_world.c

```
/*
    Seu primeiro programa em C
    ** Para dar sorte! **
*/

#include <stdio.h>           // Inclui a biblioteca padrao de
                             // entrada e saida (Standard Input/Output)

int main(void) {

    printf("Hello, World!\n"); // Exibe "Hello, World!" na tela
    return 0;

}
```

Apesar de simples, este programa já contém quase todos os elementos estruturais que qualquer programa em C possui. Vamos destrinchá-lo peça por peça.

1.1.1 Comentários

As primeiras linhas do código são um comentário de múltiplas linhas, delimitado por `/*` e `*/`. Mais adiante, na mesma linha do `#include`, aparece um comentário de linha única, iniciado por `//` e válido até o final da linha.

Sintaxe de comentários

```
/*  
    Este e' um comentario de multiplas linhas.  
    Pode ocupar quantas linhas forem necessarias.  
*/  
  
// Este e' um comentario de uma unica linha.
```

Comentários são trechos de texto totalmente ignorados pelo compilador — eles existem apenas para quem lê o código. Use-os para explicar *por que* uma decisão foi tomada, não para descrever o óbvio.

1.1.2 A diretiva `#include`

A linha `#include <stdio.h>` não é, tecnicamente, um comando C: é uma **diretiva de pré-processador**, processada *antes* da compilação propriamente dita. Ela instrui o compilador a inserir, naquele ponto do arquivo, o conteúdo do arquivo de cabeçalho (*header*) `stdio.h`, que declara as funções da biblioteca padrão de entrada e saída — entre elas, a função `printf` usada logo abaixo. O pré-processador e as bibliotecas de software serão tratados com mais profundidade no Capítulo 11.

1.1.3 A função `main`

Todo programa C possui exatamente um ponto de entrada: a função `main`. É a partir dela que a execução do programa começa.

Assinatura da função `main`

```
int main(void) {  
    // ... corpo do programa ...  
}
```

- O `int` antes de `main` indica que essa função **retorna** um número inteiro ao sistema operacional quando termina — o chamado *código de saída*;
- O `void` entre parênteses indica que, neste caso, a função não recebe parâmetros de entrada. É possível receber argumentos de linha de comando aqui, assunto tratado

mais adiante no material;

- Tudo o que está entre as chaves `{ }` constitui o **corpo** (ou escopo) da função — o bloco de instruções que será executado.

1.1.4 A instrução `printf`

`printf` (*print formatted*) é uma função da biblioteca `stdio.h` responsável por exibir texto na saída padrão — normalmente, o terminal. O texto entre aspas duplas é uma **string** (uma sequência de caracteres); a sequência `\n` dentro dela é um **caractere de escape** que representa uma quebra de linha. Formatação de entrada e saída de dados é aprofundada no Capítulo 4.

1.1.5 O retorno e o ponto e vírgula

A última instrução, `return 0;`, encerra a execução de `main` devolvendo o valor `0` ao sistema operacional — por convenção, `0` significa “o programa terminou sem erros”. Qualquer outro valor costuma indicar algum tipo de falha. Esse conceito é fundamental em engenharia de software: é assim que um programa comunica sucesso ou falha para quem o executou (outro programa, um script, ou o próprio sistema operacional).

Repare também que **toda instrução em C termina com ponto e vírgula (;)**. Esquecer esse detalhe é, disparadamente, o erro de sintaxe mais comum entre iniciantes.

O compilador C é implacável com pequenos deslizes de sintaxe: uma chave não fechada, um ponto e vírgula esquecido ou um parêntese a mais são suficientes para impedir a compilação inteira do programa. Leia as mensagens de erro com atenção — elas quase sempre indicam a linha exata (ou próxima dela) onde o problema ocorre.

1.2 Compilando e executando

Para transformar o arquivo `hello_world.c` em um programa executável, usamos um compilador. No Linux e no macOS, o mais comum é o **GCC** (*GNU Compiler Collection*); no Windows, alternativas como MinGW ou o compilador do próprio Visual Studio cumprem o mesmo papel.

Compilando com gcc

```
$ gcc -o hello_world hello_world.c
```

O parâmetro `-o hello_world` define o nome do arquivo executável gerado. Sem ele, o GCC produziria um arquivo chamado `a.out` por padrão. Para executar o programa:

Executando o programa

```
$ ./hello_world
Hello, World!
```

No Linux/macOS, o `./` antes do nome do executável é necessário porque, por segurança, o diretório atual normalmente não faz parte do `PATH` do sistema — sem ele, o shell não saberia onde procurar o programa.

1.3 E no ESP32?

Um microcontrolador como o ESP32 não tem teclado, tela ou sistema operacional convencional — então não existe um terminal esperando por `Hello, World!`. Ainda assim, o princípio é idêntico: escrevemos código C, ele é compilado, e o resultado (um arquivo *binário*) é carregado para a memória do dispositivo, onde passa a ser executado assim que ele é ligado.

No mundo dos microcontroladores, o equivalente ao *Hello, World!* costuma ser o **blink**: piscar um LED. É o primeiro programa que qualquer pessoa escreve para o ESP32, porque prova visualmente que o código foi compilado, carregado e está de fato em execução no hardware. Chegaremos lá: a partir do Capítulo 14, vamos configurar o ambiente PlatformIO e, no Capítulo 16, escrever exatamente esse programa.

A diferença mais importante a guardar por enquanto é estrutural. Você viu que o nosso programa C tradicional usa uma função `main` que roda uma vez, exibe a mensagem na tela, e termina devolvendo `0` ao sistema operacional.

Um microcontrolador funciona de forma diferente. Não faria sentido um programa finalizar sua execução — se ele acabar, o que o hardware faria depois? Por isso, a programação para o ESP32 é baseada em um **ciclo contínuo**. Em vez de uma função que termina, o código de um firmware é desenhado para rodar infinitamente, repetindo instruções e reagindo ao mundo exterior enquanto a placa estiver ligada.

Quando chegarmos na Parte II e formos programar o ESP32 usando o framework Arduino, essa estrutura aparecerá de forma bem explícita: em vez de usarmos o `main`, dividiremos nosso código em duas partes: uma função chamada `setup()` para configurações iniciais, e uma função chamada `loop()`, que o próprio sistema se encarrega de repetir para sempre.

Toda a Parte I desta apostila usa o `gcc` e a execução em um computador comum como ambiente de aprendizado, exatamente porque isso permite testar cada conceito de forma rápida e isolada, sem depender de hardware específico. Os mesmos conceitos — tipos de dados, estruturas de controle, funções, ponteiros — são diretamente aplicáveis ao firmware do ESP32 que construiremos na Parte II.

CAPÍTULO 2

Sistemas Numéricos, Memória e Compilação

Antes de avançar pela sintaxe da linguagem, vale entender melhor o terreno sobre o qual ela é construída: como um computador representa números, como organiza a memória e o que, de fato, acontece entre escrever um arquivo `.c` e obter um programa executável. Esses fundamentos são especialmente relevantes em sistemas embarcados, onde memória é escassa e o programador frequentemente manipula dados diretamente em binário e hexadecimal.

2.1 Sistema binário

Computadores armazenam e processam informação usando apenas dois estados elétricos — ligado e desligado —, representados matematicamente pelos dígitos 0 e 1. Esse é o **sistema binário** (base 2). Cada dígito binário é chamado de **bit** (*binary digit*); um grupo de 8 bits forma um **byte**.

Um número binário é interpretado da mesma forma que um número decimal, mas com potências de 2 em vez de potências de 10:

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

Bits	Nome	Valores possíveis	Intervalo (sem sinal)
1	bit	2	0 a 1
4	nibble	16	0 a 15
8	byte	256	0 a 255
16	word (na maioria dos MCUs de 16b)	65 536	0 a 65 535
32	double word	4 294 967 296	0 a $2^{32} - 1$

Tabela 2.1: Grupos comuns de bits e sua capacidade de representação.

2.2 Sistemas octal e hexadecimal

Escrever endereços de memória ou máscaras de bits inteiramente em binário é verboso e propenso a erros. Por isso, é comum usar bases que sejam potências de 2 e que reduzam o número de dígitos: o **octal** (base 8) e, principalmente, o **hexadecimal** (base 16).

No hexadecimal, os dígitos vão de 0 a 9 e depois de A a F (representando 10 a 15). Cada dígito hexadecimal representa exatamente 4 bits (um nibble), o que torna a conversão entre binário e hexadecimal praticamente mecânica:

Em C, cada base tem uma notação própria para constantes literais:

Decimal	Binário	Octal	Hexadecimal
0	0000	0	0
7	0111	7	7
10	1010	12	A
15	1111	17	F
255	1111 1111	377	FF

Tabela 2.2: Um mesmo valor em diferentes bases numéricas.

Notação de bases numéricas em C

```
int decimal    = 42;           // Base 10 (padrao)
int octal      = 052;          // Base 8  (prefixo 0)
int hexadecimal = 0x2A;         // Base 16 (prefixo 0x)
int binario    = 0b101010;     // Base 2  (prefixo 0b, extensao GNU/C23)
```

O prefixo `0` para octal é uma pegadinha clássica: escrever `int codigo = 057;` pensando em “cinquenta e sete” na verdade produz o valor 47 em decimal, porque o compilador interpreta o zero à esquerda como indicação de base octal.

Hexadecimal é a notação padrão para endereços de memória e para os **registradores** de periféricos do ESP32 (ex.: `0x3FF44004`), e é extremamente comum representar conjuntos de pinos GPIO como **máscaras de bits** em binário ou hexadecimal — cada bit ligado corresponde a um pino específico habilitado. Voltaremos a essa ideia no Capítulo 16, quando configurarmos registradores de GPIO diretamente.

Se você assistiu ao filme *Perdido em Marte* (ou leu o livro de Andy Weir), viu o sistema hexadecimal resolver um problema vital de engenharia. No enredo, o astronauta Mark Watney precisa se comunicar com a Terra usando a câmera giratória da antiga sonda [Mars Pathfinder](#).

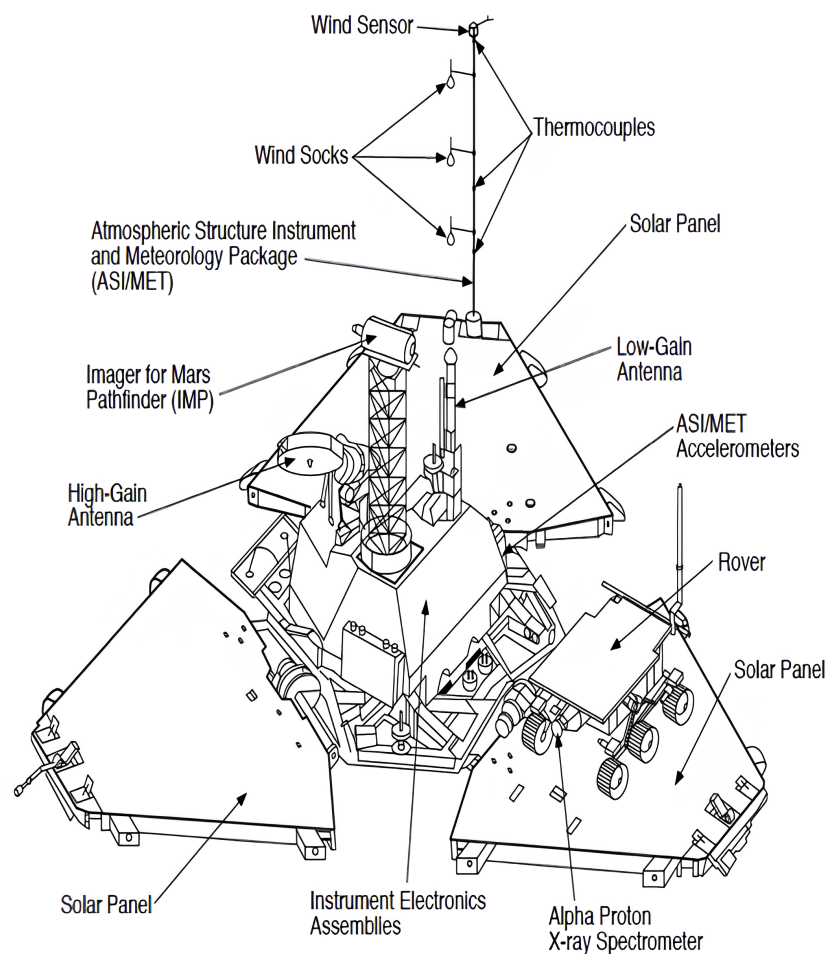


Figura 2.1: Diagrama da plataforma de pouso da missão Mars Pathfinder da NASA. Repare na câmera rotatória acoplada ao sistema.

Ele precisava que a câmera apontasse para placas com letras ao seu redor. Se usasse o alfabeto convencional (26 letras), ele teria que dividir os 360 graus do círculo por 26, o que dá fatias estreitas de cerca de 13,8 graus. A chance de a câmera apontar para a fronteira entre duas letras e gerar ambiguidade era enorme.

A solução? Usar a tabela ASCII (que mapeia texto em números) em formato **hexadecimal**. Como o hexadecimal possui apenas 16 símbolos (0 a 9 e A a F), ele pôde dividir os 360 graus por 16. O resultado foram fatias largas e seguras de 22,5 graus. Assim, para a Terra enviar a letra "H" (cujo valor na tabela ASCII é 72 em decimal, ou `0x48` em hexadecimal), a câmera girava primeiro para a placa 4 e depois para a placa 8.

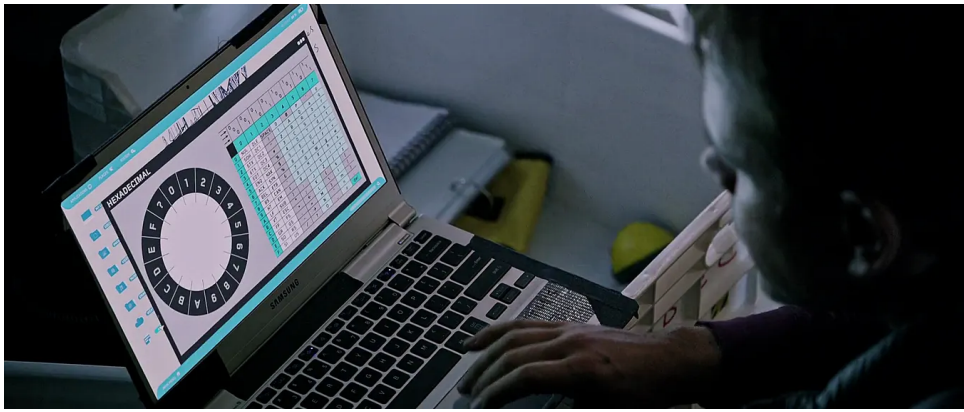


Figura 2.2: Mark Watney acessando a tabela ASCII pelo notebook de Beth Johanssen.

A cena do primeiro contato do filme pode ser vista neste [link Youtube](#).

2.3 Memória

Um programa em execução manipula dados que residem na **memória** do sistema. Em um computador de propósito geral, essa memória é tipicamente volátil (RAM) e vasta (gigabytes); o sistema operacional gerencia sua alocação de forma praticamente transparente ao programador.

Um microcontrolador como o ESP32 tem uma realidade bem diferente:

- **Flash** — memória não volátil (mantém os dados sem energia), onde o firmware compilado é gravado. No ESP32, tipicamente 4 MB ou mais;
- **SRAM** — memória volátil, rápida, usada para variáveis, pilha (*stack*) e heap durante a execução. No ESP32, apenas algumas centenas de kB — ordens de magnitude menor que a RAM de um computador comum.

Essa escassez de memória é a razão pela qual, em programação embarcada, escolher o **tipo de dado** correto para cada variável (assunto do Capítulo 3) deixa de ser um preciosismo e passa a ser uma decisão de engenharia: usar um `int` de 32 bits para armazenar um valor que cabe perfeitamente em 8 bits desperdiça memória que, em um sistema embarcado, pode ser exatamente o que falta.

2.4 Pré-processamento

Antes que o compilador propriamente dito veja o código, ele passa por um estágio chamado **pré-processamento**, controlado por linhas que começam com `#` — as **diretivas**. Já vimos uma delas, `#include`. Outras diretivas comuns:

Diretivas de pré-processador

```
#include <stdio.h>           // inclui um cabeçalho da biblioteca padrao
#include "meu_arquivo.h"      // inclui um cabeçalho do proprio projeto
```

```
#define PI 3.14159           // define uma macro/constante simbolica
#define QUADRADO(x) ((x) * (x)) // macro com "parametro"

#ifdef DEBUG
    printf("Modo debug ativo\n");
#endif
```

A diretiva `#define` cria uma **macro**: onde quer que `PI` apareça no código depois dessa linha, o pré-processador o substitui literalmente por `3.14159`, antes mesmo de o compilador analisar a sintaxe. Diretivas condicionais como `#ifdef`/`#endif` permitem incluir ou excluir trechos inteiros de código dependendo de configurações — um recurso muito usado em projetos embarcados para, por exemplo, ativar mensagens de depuração apenas em builds de desenvolvimento.

Veja como isso se aplica na prática em um código para o ESP32. Podemos criar uma macro chamada `DEBUG` e usá-la para "ligar" ou "desligar" o envio de dados pela porta serial:

Depuração condicional com macros

```
#define DEBUG // Comente esta linha (coloque // na frente) para desativar o debug

void setup() {
    // Inicializa a serial (nao tem problema inicializar mesmo sem usar)
    Serial.begin(115200);

    #ifdef DEBUG
        Serial.println("Sistema iniciado. Modo DEBUG ativo.");
    #endif
}

void loop() {
    int leitura = analogRead(34); // Leitura de um sensor qualquer

    #ifdef DEBUG
        Serial.print("Valor lido pelo sensor: ");
        Serial.println(leitura);
    #endif

    // A logica principal do programa continua aqui,
    // sem interrupcoes da serial se o DEBUG estiver desativado
    delay(1000);
}
```

Essa técnica é extremamente poderosa. Ela permite que você encha o seu código de mensagens úteis para investigar o comportamento do sistema durante o desenvolvimento. Quando o projeto estiver pronto para uso real, basta comentar a primeira linha (`// #define DEBUG`). O pré-processador, então, apagará invisivelmente tudo o que estiver entre os blocos `#ifdef DEBUG` e `#endif` antes da compilação. O resultado? Um firmware final menor, mais rápido e que não desperdiça processamento enviando texto para uma porta USB vazia.

2.5 Compilação

O termo “compilar” esconde, na verdade, quatro etapas distintas:

1. **Pré-processamento** — expande `#include`, `#define` e diretivas condicionais, produzindo um único arquivo de código-fonte “puro”;
2. **Compilação** — traduz o código C em código assembly equivalente para a arquitetura de destino;
3. **Montagem (assembly)** — converte o código assembly em código de máquina, gerando um arquivo objeto (`.o`);
4. **Ligação (linking)** — combina o(s) arquivo(s) objeto com as bibliotecas necessárias (como a implementação de `printf`) em um único executável.

O `gcc` executa essas quatro etapas automaticamente, mas é possível observá-las de forma isolada com a seguir:

Etapas de compilação separadas

```
$ gcc -E hello_world.c -o hello_world.i    # so o pre-processor
$ gcc -S hello_world.i -o hello_world.s    # gera assembly
$ gcc -c hello_world.s -o hello_world.o    # gera arquivo objeto
$ gcc hello_world.o -o hello_world        # linka e gera o executavel
```

Ao compilar um projeto para o ESP32 no PlatformIO, essas mesmas quatro etapas acontecem — só que usando um compilador **cruzado** (*cross-compiler*): o código roda no seu computador (arquitetura x86/ARM), mas o binário gerado é destinado a um processador **Xtensa** (a arquitetura do ESP32), completamente diferente. O PlatformIO baixa e gerencia essa toolchain de compilação cruzada automaticamente — é uma das razões pelas quais ele simplifica tanto o processo. Detalhes no Capítulo 14.

2.6 Números pseudoaleatórios

Muitos programas — jogos, simulações, e também firmwares embarcados — precisam de números aparentemente aleatórios. Como um computador é uma máquina determinística, ele não gera aleatoriedade verdadeira a partir do nada: gera números **pseudoaleatórios**, calculados por uma fórmula a partir de um valor inicial chamado **semente** (*seed*).

Gerando números pseudoaleatórios

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

```
int main(void) {  
  
    srand(time(NULL));          // inicializa a semente com o tempo atual  
    int dado = (rand() % 6) + 1; // numero pseudoaleatorio entre 1 e 6  
  
    printf("Voce tirou: %d\n", dado);  
    return 0;  
}
```

A função `srand` define a semente do gerador; usar `time(NULL)` (o tempo atual em segundos) garante uma semente diferente a cada execução. A função `rand()` retorna um valor inteiro pseudoaleatório; o operador `%` (resto da divisão) é usado para restringir o resultado a um intervalo específico — no exemplo, de 1 a 6, como em um dado de tabuleiro.

`rand()` **não** é adequado para fins criptográficos — sua sequência é previsível se a semente for conhecida. Para geração de números aleatórios seguros, bibliotecas específicas de criptografia devem ser usadas.

CAPÍTULO 3

Tipos de Variáveis e Operadores

Uma **variável** é um espaço nomeado na memória usado para armazenar um valor que pode mudar ao longo da execução do programa. Diferente de linguagens como Python, C exige que o **tipo** de cada variável seja declarado explicitamente (tipagem estática) — é isso que permite ao compilador saber, entre outras coisas, quantos bytes reservar para ela.

3.1 Tipos primitivos

Tipo	Uso típico	Tamanho comum*
<code>char</code>	um caractere / inteiro pequeno	1 byte
<code>int</code>	número inteiro	4 bytes
<code>float</code>	número em ponto flutuante (precisão simples)	4 bytes
<code>double</code>	número em ponto flutuante (precisão dupla)	8 bytes
<code>bool</code> [†]	valor lógico (verdadeiro/falso)	1 byte

Tabela 3.1: Tipos primitivos da linguagem C. *O tamanho exato depende da arquitetura e do compilador — nunca assuma, use `sizeof`. [†]`bool` requer `#include <stdbool.h>`.

Os tipos `int`, `char` e `double` ainda podem ser modificados por qualificadores:

Qualificadores de tipo

```
short int  a;  // inteiro "curto" (tipicamente 2 bytes)
long int   b;  // inteiro "longo" (tipicamente 4 ou 8 bytes)
long long  c;  // inteiro "bem longo" (tipicamente 8 bytes)

unsigned int d; // apenas valores nao-negativos (dobra o limite superior)
signed int   e; // admite valores negativos (comportamento padrao de "int")
```

Por padrão, os tipos inteiros são `signed` (aditem valores negativos, usando uma representação chamada complemento de dois). Adicionar `unsigned` descarta o sinal e usa todos os bits para magnitude — um `uint8_t` (8 bits sem sinal) vai de 0 a 255; sua versão com sinal, de -128 a 127.

Descobrimo o tamanho de um tipo

```
#include <stdio.h>

int main(void) {
```

```
printf("sizeof(int)      = %zu bytes\n", sizeof(int));
printf("sizeof(double) = %zu bytes\n", sizeof(double));
printf("sizeof(char)    = %zu bytes\n", sizeof(char));
return 0;
}
```

O operador `sizeof` retorna, em tempo de compilação, o número de bytes ocupados por um tipo ou variável — é a ferramenta certa para nunca precisar “chutar” o tamanho de nada.

A linguagem C **garante apenas tamanhos mínimos** para os tipos primitivos, não tamanhos exatos — `int` pode ter 2, 4 ou até 8 bytes dependendo da plataforma e do compilador. Código que assume um tamanho fixo para `int` pode compilar perfeitamente em um computador e se comportar de forma diferente (ou quebrar) em um microcontrolador. É exatamente esse problema que a próxima seção resolve.

3.2 Tipos de largura fixa: `stdint.h`

Para eliminar essa ambiguidade — crítica em sistemas embarcados, onde o tamanho exato de cada variável impacta diretamente no uso de uma memória escassa —, a biblioteca padrão `<stdint.h>` define tipos com largura **garantida**:

Tipo	Descrição
<code>int8_t</code> / <code>uint8_t</code>	inteiro de 8 bits, com/sem sinal (0 a 255)
<code>int16_t</code> / <code>uint16_t</code>	inteiro de 16 bits, com/sem sinal (0 a 65 535)
<code>int32_t</code> / <code>uint32_t</code>	inteiro de 32 bits, com/sem sinal (0 a $2^{32}-1$)
<code>int64_t</code> / <code>uint64_t</code>	inteiro de 64 bits, com/sem sinal

Tabela 3.2: Tipos de largura fixa de `<stdint.h>`.

No código de firmware para o ESP32 (e em praticamente todo código embarcado sério), `uint8_t`, `uint16_t` e `uint32_t` são usados com muito mais frequência do que `int` puro. O motivo é duplo: primeiro, previsibilidade — os registradores dos periféricos do chip exigem tamanhos exatos; segundo, economia de memória — se um valor de sensor nunca ultrapassa 255, guardá-lo em `uint8_t` em vez de `int` (que pode ocupar o quádruplo do espaço) é uma escolha consciente de engenharia, não um exagero.

3.3 Operadores

3.3.1 Aritméticos

Operador	Significado	Exemplo
+	adição	<code>a + b</code>
-	subtração	<code>a - b</code>
*	multiplicação	<code>a * b</code>
/	divisão	<code>a / b</code>
%	resto da divisão (módulo)	<code>a % b</code>

Tabela 3.3: Operadores aritméticos.

A divisão entre dois inteiros em C é sempre **inteira** — o resultado é truncado, não arredondado. `7 / 2` vale `3`, não `3.5`. Para obter um resultado fracionário, pelo menos um dos operandos precisa ser de ponto flutuante: `7.0 / 2` vale `3.5`.

3.3.2 Relacionais e lógicos

Relacionais		Lógicos	
<code>==</code>	igual a	<code>&&</code>	E lógico (<i>and</i>)
<code>!=</code>	diferente de	<code> </code>	OU lógico (<i>or</i>)
<code>></code> <code><</code>	maior / menor que	<code>!</code>	NÃO lógico (<i>not</i>)
<code>>=</code> <code><=</code>	maior/menor ou igual		

Tabela 3.4: Operadores relacionais e lógicos — o resultado é sempre 0 (falso) ou 1 (verdadeiro).

Um dos erros mais clássicos em C é confundir o operador de **atribuição** (`=`) com o de **comparação** (`==`). A expressão `if (x = 5)` é *sintaticamente válida*: ela atribui 5 a `x` e depois avalia esse resultado (sempre verdadeiro, pois 5 é diferente de zero) — quase nunca é isso que se pretendia escrever.

3.3.3 Bit a bit (*bitwise*)

Operadores bit a bit atuam diretamente sobre a representação binária dos operandos, bit por bit — e são especialmente importantes em programação embarcada.

Operador	Significado	Exemplo
&	E bit a bit (AND)	0b1100 & 0b1010 == 0b1000
	OU bit a bit (OR)	0b1100 0b1010 == 0b1110
^	OU exclusivo (XOR)	0b1100 ^ 0b1010 == 0b0110
~	complemento (NOT)	~0b0000 == 0b1111
<<	deslocamento à esquerda	1 << 3 == 0b1000 (=8)
>>	deslocamento à direita	8 >> 2 == 0b0010 (=2)

Tabela 3.5: Operadores bit a bit.

Operadores bit a bit são a ferramenta padrão para manipular **registradores** de periféricos e criar **máscaras de bits** — muito usadas para representar, de forma compacta, o estado de vários pinos ou dispositivos digitais em uma única variável. Por exemplo, o estado de 8 relés (ligado/desligado) cabe inteiro em um único `uint8_t`, um bit por relé:

```
uint8_t estado_reles = 0;

estado_reles |= (1 << 3); // liga o rele numero 3
estado_reles &= ~(1 << 5); // desliga o rele numero 5

if (estado_reles & (1 << 3)) { // verifica se o rele 3 esta ligado
    printf("Rele 3 esta ligado\n");
}
```

O operador `<<` desloca o bit 1 para a posição correspondente ao dispositivo desejado; `|=` liga esse bit sem afetar os demais; `&= ~(...)` desliga apenas ele. Voltaremos a essa ideia de forma prática ao ler e escrever pinos digitais no Capítulo 16.

3.3.4 Atribuição composta

Operadores de atribuição composta

```
int contador = 0;

contador += 5; // equivalente a: contador = contador + 5;
contador -= 2; // equivalente a: contador = contador - 2;
contador *= 3; // equivalente a: contador = contador * 3;
contador /= 2; // equivalente a: contador = contador / 2;

contador++; // incrementa em 1 (equivalente a contador += 1)
contador--; // decrementa em 1 (equivalente a contador -= 1)
```

`contador++` (pós-incremento) e `++contador` (pré-incremento) têm o mesmo efeito *sobre a variável*, mas diferem no *valor retornado pela expressão* quando usados dentro de outra operação — por exemplo, `y = x++` atribui a `y` o valor *antigo* de `x` e só depois incrementa; `y = ++x` incrementa primeiro e atribui o novo valor. Quando o incremento é usado sozinho, em sua própria linha, essa diferença não importa.

3.4 Conversão de tipos (*casting*)

Quando uma expressão mistura tipos diferentes, C converte automaticamente os valores para um tipo comum antes de operar — a chamada **conversão implícita**. Por exemplo, a operação `7.0 / 2` funciona porque o `2` (inteiro) é implicitamente convertido para `2.0` (ponto flutuante) antes da divisão.

Também é possível forçar uma conversão explicitamente, escrevendo o tipo desejado entre parênteses antes da expressão — o chamado **cast**:

Conversão explícita de tipos

```
#include <stdio.h>

int main(void) {
    int total = 7, quantidade = 2;

    float media_errada = total / quantidade;           // 3.0 (divisao inteira
    ↪ antes!)
    float media_certa  = (float) total / quantidade;   // 3.5 (cast antes da
    ↪ divisao)

    printf("%.1f\n", media_errada);
    printf("%.1f\n", media_certa);

    return 0;
}
```

Converter de um tipo “maior” para um “menor” (por exemplo, `double` para `int`, ou `int` para `uint8_t`) pode **truncar** ou **perder informação**: `(int) 3.99` vale 3, não 4 (a parte fracionária é simplesmente descartada, sem arredondar); atribuir o valor 300 a uma variável `uint8_t` (que só representa até 255) produz um resultado *enrolado* (*overflow*), não um erro — o compilador, em geral, nem avisa.

Casts explícitos aparecem com frequência no código de firmware, principalmente ao converter a leitura bruta e inteira de um sensor analógico para uma grandeza física em

ponto flutuante. É exatamente o que faremos no Capítulo 17: converter o valor devolvido por `analogRead()` (um `int` entre 0 e 4095) em uma tensão em volts, algo como `(float) leitura / 4095.0f * 3.3f`.

CAPÍTULO 4

Entrada e Saída de Dados

Um programa que não troca informação com o mundo exterior tem utilidade limitada. Nesta seção vemos como C lê dados fornecidos pelo usuário (entrada) e como exibe resultados (saída), usando as funções da biblioteca padrão `<stdio.h>`.

4.1 Saída formatada: `printf`

Já usamos `printf` para exibir texto fixo. Seu verdadeiro poder está nos **especificadores de formato**, que permitem inserir valores de variáveis dentro da string de saída.

Especificadores de formato

```
#include <stdio.h>

int main(void) {
    int idade = 25;
    float altura = 1.78f;
    char inicial = 'A';

    printf("Idade: %d anos\n", idade);
    printf("Altura: %.2f m\n", altura);
    printf("Inicial: %c\n", inicial);
    printf("Idade em hex: %x, em octal: %o\n", idade, idade);

    return 0;
}
```

Especificador	Tipo
%d / %i	inteiro com sinal (<code>int</code>)
%u	inteiro sem sinal (<code>unsigned int</code>)
%f	ponto flutuante (<code>float</code> / <code>double</code>)
%c	caractere único (<code>char</code>)
%s	string (<code>char *</code>)
%x / %X	inteiro em hexadecimal (minúsculo/maiúsculo)
%o	inteiro em octal
%p	endereço de ponteiro
%%	imprime o caractere % literal

Tabela 4.1: Principais especificadores de formato de `printf`/`scanf`.

O especificador pode incluir modificadores de precisão e largura: `%.2f` imprime um ponto flutuante com duas casas decimais; `%5d` reserva um campo de 5 caracteres de largura, alinhado à direita.

4.2 Entrada formatada: `scanf`

A função `scanf` lê dados digitados pelo usuário no terminal, seguindo a mesma lógica de especificadores de `printf` — porém exige o **endereço** da variável onde o valor lido será armazenado, indicado pelo operador `&` (introduzido formalmente no Capítulo 9).

Lendo dados do teclado

```
#include <stdio.h>

int main(void) {
    int idade;

    printf("Digite sua idade: ");
    scanf("%d", &idade);

    printf("Voce tem %d anos.\n", idade);
    return 0;
}
```

Esquecer o `&` antes da variável em `scanf` é um erro comum e perigoso: em vez do endereço da variável, `scanf` recebe o *valor* nela contido, interpretado como se fosse um endereço de memória — e tenta escrever ali. O compilador frequentemente emite um aviso (*warning*), mas o programa pode compilar mesmo assim e falhar (ou pior, corromper memória) apenas durante a execução.

`scanf` é útil para aprender, mas tem limitações importantes de segurança e de tratamento de erros (não valida corretamente o que foi digitado, e é fácil causar *buffer overflow* ao ler strings sem limitar o tamanho). Em código de produção, funções como `fgets` combinada com `sscanf` costumam ser preferidas.

4.3 Caracteres e strings

Uma *string* em C não é um tipo primitivo — é, na prática, um vetor de caracteres (`char[]`) terminado por um caractere nulo especial, `'\0'`, que marca seu fim. Esse assunto será retomado com mais profundidade no Capítulo 8.

Strings como vetores de caracteres

```
#include <stdio.h>

int main(void) {
    char nome[20];

    printf("Digite seu nome: ");
    scanf("%19s", nome); // le ate 19 caracteres (+ '\0' automatico)

    printf("Ola, %s!\n", nome);
    return 0;
}
```

4.4 Sequências de escape

Dentro de strings e caracteres literais, a barra invertida introduz caracteres especiais que não têm representação visual direta:

Sequência	Significado
<code>\n</code>	nova linha (<i>newline</i>)
<code>\t</code>	tabulação horizontal
<code>\r</code>	retorno de carro (<i>carriage return</i>)
<code>\0</code>	caractere nulo (fim de string)
<code>\\</code>	barra invertida literal
<code>\"</code>	aspas duplas literais (dentro de uma string)

Tabela 4.2: Sequências de escape mais comuns.

4.5 Manipulação de arquivos

A biblioteca `<stdio.h>` também fornece funções para ler e escrever arquivos em disco, usando um ponteiro do tipo `FILE *` como identificador do arquivo aberto.

Escrevendo e lendo um arquivo de texto

```
#include <stdio.h>

int main(void) {
    FILE *arquivo = fopen("dados.txt", "w");
    if (arquivo == NULL) {
        printf("Erro ao abrir o arquivo!\n");
        return 1;
    }

    fprintf(arquivo, "Temperatura: %d C\n", 27);
}
```

```
fclose(arquivo);  
  
return 0;  
}
```

O padrão `"w"` abre o arquivo para escrita (criando-o se não existir, ou apagando seu conteúdo se existir); `"r"` abre para leitura; `"a"` adiciona conteúdo ao final de um arquivo existente. Verificar se `fopen` retornou `NULL` é essencial — é assim que a função sinaliza falha (por exemplo, permissão negada ou disco cheio).

4.6 Saída como ferramenta de depuração

Além de comunicar resultados ao usuário final, a saída de um programa é, na prática, a ferramenta de **depuração** (*debug*) mais usada por programadores iniciantes e experientes — imprimir o valor de uma variável em pontos estratégicos do código para entender *por que* ele não está se comportando como esperado é uma técnica tão comum que tem até um apelido: *print debugging*.

Depurando um cálculo com printf

```
#include <stdio.h>  
  
int main(void) {  
    int total = 0;  
  
    for (int i = 1; i <= 5; i++) {  
        total += i;  
        printf("[DEBUG] i = %d, total acumulado = %d\n", i, total);  
    }  
  
    printf("Resultado final: %d\n", total);  
    return 0;  
}
```

Um prefixo consistente, como `[DEBUG]` no exemplo acima, ajuda a distinguir mensagens de depuração (que normalmente serão removidas ou desativadas depois) das mensagens de saída “de verdade” do programa. Ferramentas de depuração mais sofisticadas (como o *debugger* `gdb`, capaz de pausar a execução e inspecionar variáveis sem precisar de nenhum `printf`) existem e são valiosas, mas a técnica de espalhar mensagens de saída pelo código continua sendo, de longe, a mais rápida e mais usada no dia a dia — especialmente em sistemas embarcados, como veremos a seguir.

Repare que o ESP32 não tem, por padrão, teclado nem tela — mas tem algo funcionalmente equivalente: a **porta serial** (UART), conectada ao computador via USB. No framework Arduino,

a saída não passa pelo `printf` da biblioteca padrão diretamente: usa-se o objeto `Serial`, inicializado uma única vez em `setup()` com `Serial.begin(115200);`. A partir daí, utiliza-se `Serial.println("mensagem")` que exibe uma linha de texto, e `Serial.printf("valor: %d\n", x)` aceita exatamente os **mesmos especificadores de formato** de `printf` apresentados neste capítulo — o conhecimento é diretamente reaproveitável, só muda a forma de chamar a função. É assim que toda a técnica de depuração por mensagens de saída, vista acima, se transporta para o firmware: em vez de rodar em um terminal, as mensagens `[DEBUG]` aparecem no **monitor serial**. O Capítulo 18 é dedicado inteiramente a essa prática no ESP32.

Já `scanf` praticamente não tem uso em firmware embarcado — a “entrada” normalmente vem de sensores e botões, não de um teclado, como veremos a partir do Capítulo 16.

Quanto a arquivos: o ESP32 pode montar um sistema de arquivos dentro da própria memória flash (usando bibliotecas como LittleFS, também disponíveis no framework Arduino), permitindo abrir, ler e escrever arquivos com uma API muito parecida com a de `<stdio.h>` — extremamente útil para salvar configurações ou registros que precisam sobreviver a um desligamento.

CAPÍTULO 5

Estruturas de Decisão

Até aqui, todo código apresentado executa suas instruções sequencialmente, uma após a outra, do início ao fim. Estruturas de decisão permitem que o programa escolha *qual* caminho seguir, dependendo de uma condição — é isso que dá a um programa a capacidade de reagir de forma diferente a diferentes situações.

5.1 if / else

Estrutura if / else if / else

```
#include <stdio.h>

int main(void) {
    int temperatura = 32;

    if (temperatura > 30) {
        printf("Esta quente!\n");
    } else if (temperatura < 15) {
        printf("Esta frio!\n");
    } else {
        printf("Temperatura agradável.\n");
    }

    return 0;
}
```

Em C, **qualquer valor diferente de zero é considerado verdadeiro**; apenas o valor `0` é considerado falso. Isso significa que `if (10)` sempre executa seu bloco, e `if (0)` nunca executa — mesmo sem envolver nenhum operador relacional explícito.

Quando o bloco de um `if` (ou de qualquer estrutura de controle) contém uma única instrução, as chaves são opcionais. Ainda assim, é considerada boa prática sempre usá-las — isso evita um erro clássico onde uma segunda linha, adicionada depois por engano fora das chaves, passa a ser executada incondicionalmente.

5.2 Operador ternário

Para condicionais simples que retornam um valor, C oferece uma forma compacta:

Operador ternário

```
int a = 10, b = 20;
int maior = (a > b) ? a : b;    // "se a > b, entao a, senao b"

printf("O maior valor e %d\n", maior);
```

5.3 switch

Quando uma mesma variável precisa ser comparada contra vários valores possíveis, o `switch` costuma ser mais legível do que uma longa cadeia de `else if`:

Estrutura switch

```
#include <stdio.h>

int main(void) {
    int dia = 3;

    switch (dia) {
        case 1:
            printf("Segunda-feira\n");
            break;
        case 2:
            printf("Terca-feira\n");
            break;
        case 3:
            printf("Quarta-feira\n");
            break;
        default:
            printf("Dia invalido\n");
    }

    return 0;
}
```

Esquecer o `break` ao final de um `case` faz a execução “cair” (*fall-through*) para o próximo `case`, executando-o também — mesmo que sua condição não tenha sido satisfeita. Às vezes esse comportamento é usado intencionalmente (para agrupar vários casos com a mesma ação), mas na grande maioria das vezes é um bug. O `default`, por convenção o último caso, captura qualquer valor não tratado pelos `cases` anteriores.

Estruturas `switch` são extremamente comuns em firmware para implementar **máquinas de estado** — por exemplo, controlar as diferentes fases de operação de um dispositivo (“aguardando”, “conectando ao Wi-Fi”, “lendo sensor”, “enviando dados”, “erro”), onde

cada estado é representado por um valor de um `enum` (Capítulo 10) e o `switch` decide o que fazer em cada um deles a cada iteração do laço principal do programa.

CAPÍTULO 6

Laços de Repetição e Desvios

Laços (ou *loops*) permitem repetir um bloco de instruções múltiplas vezes, sem precisar reescrevê-lo. C oferece três estruturas de repetição — `for`, `while` e `do-while` — cada uma mais adequada a um cenário específico.

6.1 for

O laço `for` é ideal quando o número de repetições é conhecido (ou calculável) de antemão.

Laço for

```
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 5; i++) {
        printf("Iteracao %d\n", i);
    }
    return 0;
}
```

A estrutura `for` (`inicialização`; `condição`; `incremento`) tem três partes, todas opcionais:

- **Inicialização** — executada uma única vez, antes da primeira iteração (tipicamente declara e inicializa a variável de controle);
- **Condição** — testada *antes* de cada iteração; o laço continua enquanto ela for verdadeira;
- **Incremento** — executado ao final de cada iteração, antes de testar a condição novamente.

6.2 while

O laço `while` repete um bloco *enquanto* uma condição permanecer verdadeira, testada sempre no início de cada iteração — inclusive antes da primeira. É a estrutura certa quando o número de repetições não é conhecido de antemão.

Laço while

```
#include <stdio.h>

int main(void) {
    int contador = 5;
```

```
while (contador > 0) {  
    printf("Contagem: %d\n", contador);  
    contador--;  
}  
printf("Fim!\n");  
  
return 0;  
}
```

6.3 do-while

O laço `do-while` é semelhante ao `while`, mas testa a condição *depois* de cada iteração — garantindo que o bloco execute **ao menos uma vez**, mesmo que a condição já comece falsa. Isso o torna a estrutura natural para validação de entrada, onde é preciso pedir o dado ao menos uma vez antes de poder avaliar se ele é válido:

Laço do-while

```
#include <stdio.h>  
  
int main(void) {  
    int senha, tentativas = 0;  
  
    do {  
        printf("Digite a senha: ");  
        scanf("%d", &senha);  
        tentativas++;  
    } while (senha != 1234 && tentativas < 3);  
  
    if (senha == 1234) {  
        printf("Acesso liberado!\n");  
    } else {  
        printf("Numero de tentativas excedido.\n");  
    }  
  
    return 0;  
}
```

Todo firmware embarcado gira, conceitualmente, em torno de um **laço principal infinito** — algo como `while (1) { ... }`, que nunca termina enquanto o dispositivo estiver ligado, lendo sensores e atualizando saídas indefinidamente. No framework Arduino, como veremos no Capítulo 15, esse laço já vem pronto: em vez de escrevê-lo você mesmo, basta colocar o código a repetir dentro de uma função chamada `loop()` — o próprio framework se encarrega de chamá-la repetidamente, para sempre, por trás dos panos.

6.4 break e continue

Dois comandos permitem alterar o fluxo normal de um laço:

- `break` — interrompe o laço imediatamente, saindo dele por completo;
- `continue` — pula o restante da iteração atual e avança direto para a próxima.

break e continue

```
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 10; i++) {

        if (i % 2 == 0) {
            continue; // pula os numeros pares
        }

        if (i > 7) {
            break;     // interrompe o laco ao passar de 7
        }

        printf("%d\n", i); // imprime: 1, 3, 5, 7
    }
    return 0;
}
```


CAPÍTULO 7

Funções e Protótipos

Uma **função** é um bloco de código nomeado, reutilizável, que realiza uma tarefa específica. Já usamos várias — `printf`, `scanf`, `main` — mas até agora sempre prontas, fornecidas por bibliotecas. Esta seção mostra como criar as suas próprias.

7.1 Declarando e chamando funções

Uma função simples

```
#include <stdio.h>

int soma(int a, int b) {
    int resultado = a + b;
    return resultado;
}

int main(void) {
    int total = soma(5, 3);
    printf("A soma é %d\n", total);
    return 0;
}
```

Uma definição de função tem quatro partes:

- **Tipo de retorno** (`int`) — o tipo do valor devolvido por `return`. Use `void` quando a função não retorna nenhum valor;
- **Nome** (`soma`) — usado para chamá-la em outros pontos do código;
- **Parâmetros** (`int a, int b`) — as variáveis que a função recebe como entrada, entre parênteses;
- **Corpo** — o bloco de instruções entre chaves, executado a cada chamada.

7.2 Passagem de parâmetros por valor

Em C, argumentos são passados para funções **por valor**: a função recebe uma *cópia* do dado, não o dado original. Alterações feitas dentro da função não afetam a variável usada na chamada.

Passagem por valor

```
#include <stdio.h>

void dobrar(int x) {
    x = x * 2;    // altera apenas a copia local
}

int main(void) {
    int numero = 10;
    dobrar(numero);
    printf("%d\n", numero);    // ainda imprime 10!
    return 0;
}
```

Para que uma função possa modificar uma variável do código que a chamou, é preciso passar seu **endereço** (um ponteiro) em vez do valor — técnica detalhada no Capítulo 9.

7.3 Protótipos de funções

Em C, uma função precisa ser **conhecida** pelo compilador antes de ser usada. Quando a definição completa de uma função vem depois do ponto onde ela é chamada (ou está em outro arquivo), é preciso declarar um **protótipo**: a assinatura da função, sem o corpo, terminada em ponto e vírgula.

Protótipo declarado antes do uso

```
#include <stdio.h>

int soma(int a, int b);    // prototipo

int main(void) {
    printf("%d\n", soma(4, 6));    // ja pode ser chamada aqui
    return 0;
}

int soma(int a, int b) {    // definicao completa, depois do uso
    return a + b;
}
```

Protótipos são a razão de existir dos arquivos de cabeçalho (**.h**): eles reúnem as assinaturas das funções de um módulo, permitindo que outros arquivos **.c** as usem sem precisar ver (ou recompilar) sua implementação completa. Esse padrão é explorado a fundo no Capítulo 11.

7.4 Escopo de variáveis

O **escopo** de uma variável é a região do código onde ela existe e pode ser acessada. Variáveis declaradas dentro de uma função (ou de qualquer bloco `{ }`) são **locais** — existem apenas enquanto aquele bloco está em execução, e são invisíveis fora dele. Variáveis declaradas fora de qualquer função são **globais** — visíveis (e modificáveis) por todas as funções do arquivo.

Escopo local vs. global

```
#include <stdio.h>

int contador_global = 0; // variavel global

void incrementar(void) {
    int passo = 1;        // variavel local a "incrementar"
    contador_global += passo;
}

int main(void) {
    incrementar();
    incrementar();
    printf("%d\n", contador_global); // imprime 2
    // printf("%d\n", passo);        // ERRO: "passo" nao existe aqui
    return 0;
}
```

Variáveis globais são convenientes, mas usadas em excesso tornam o código difícil de entender e depurar — qualquer função pode alterá-las a qualquer momento, criando dependências ocultas. Prefira sempre passar dados explicitamente por parâmetros e devolvê-los por `return` quando possível.

7.4.1 Variáveis locais `static`

Uma variável local declarada com o qualificador `static` tem um comportamento híbrido: continua visível apenas dentro da função onde foi declarada (como qualquer variável local), mas seu valor **persiste entre chamadas** — em vez de ser recriada do zero a cada chamada da função, ela é inicializada uma única vez, na primeira execução, e mantém seu valor de uma chamada para a próxima.

Uma variável `static` mantém seu valor entre chamadas

```
#include <stdio.h>

void contar_chamadas(void) {
    static int chamadas = 0; // inicializada uma unica vez
    chamadas++;
}
```

```
    printf("Esta funcao ja foi chamada %d vez(es)\n", chamadas);  
}  
  
int main(void) {  
    contar_chamadas(); // "1 vez(es)"  
    contar_chamadas(); // "2 vez(es)"  
    contar_chamadas(); // "3 vez(es)"  
    return 0;  
}
```

Isso é útil sempre que uma função precisa “lembrar” algo de sua execução anterior — um contador de eventos, o último valor processado — sem recorrer a uma variável global.

Em firmware embarcado, algumas variáveis globais são difíceis de evitar por completo — por exemplo, uma variável compartilhada entre a rotina principal e uma **interrupção** (Capítulo 19), que roda de forma assíncrona e precisa sinalizar eventos para o resto do programa. Nesses casos, a variável costuma ser declarada com o qualificador `volatile`, que instrui o compilador a nunca otimizar (ou assumir como constante) o acesso àquela variável — porque seu valor pode mudar a qualquer momento, fora do fluxo normal do código.

CAPÍTULO 8

Arranjos: Vetores

Um **arranjo** (*array*), ou **vetor**, é uma coleção de elementos do mesmo tipo, armazenados em posições de memória contíguas e acessados por um índice numérico. É a estrutura de dados mais fundamental para trabalhar com múltiplos valores relacionados — uma lista de leituras de sensor, o histórico de temperaturas de um dia, as últimas N amostras de um sinal.

8.1 Declarando e percorrendo um vetor

Declarando e usando um vetor

```
#include <stdio.h>

int main(void) {
    int temperaturas[5] = {21, 23, 19, 25, 22};

    for (int i = 0; i < 5; i++) {
        printf("Dia %d: %d C\n", i, temperaturas[i]);
    }

    return 0;
}
```

Pontos essenciais sobre vetores em C:

- **Índices começam em 0** — o primeiro elemento de `temperaturas` é `temperaturas[0]`, e o último, em um vetor de 5 elementos, é `temperaturas[4]`;
- O **tamanho é fixo** após a declaração — um vetor declarado com 5 posições sempre ocupará espaço para 5 elementos; não é possível “crescer” um vetor comum em tempo de execução (para isso, veja alocação dinâmica no Capítulo 9);
- C **não verifica limites automaticamente**: acessar `temperaturas[10]` em um vetor de 5 posições não gera um erro imediato — apenas lê (ou escreve) memória fora da área reservada, um comportamento indefinido e uma fonte clássica de bugs graves.

Esse tipo de acesso fora dos limites (*buffer overflow*) é uma das causas mais comuns de falhas de segurança em software escrito em C. Sempre garanta, por construção do seu código, que o índice usado está dentro do intervalo válido do vetor.

8.2 Vetores e strings

Como visto no Capítulo 4, uma string em C é simplesmente um vetor de `char` terminado pelo caractere nulo `'\0'`:

Uma string é um vetor de char

```
#include <stdio.h>

int main(void) {
    char saudacao[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
    // equivalente, e muito mais pratico, a:
    char saudacao2[] = "Hello";

    printf("%s\n", saudacao2);
    return 0;
}
```

8.3 Passando arranjos para funções

Quando um vetor é passado como argumento para uma função, o que de fato é passado é um **ponteiro** para seu primeiro elemento — não uma cópia de todos os seus dados. Por isso, é comum (e necessário) passar também o tamanho do vetor como um parâmetro separado, já que a função não tem como saber onde ele termina apenas olhando para o ponteiro.

Vetor como parâmetro de função

```
#include <stdio.h>

float media(int vetor[], int tamanho) {
    int soma = 0;
    for (int i = 0; i < tamanho; i++) {
        soma += vetor[i];
    }
    return (float) soma / tamanho;
}

int main(void) {
    int leituras[4] = {30, 32, 28, 31};
    printf("Media: %.1f\n", media(leituras, 4));
    return 0;
}
```

Vetores são onipresentes em firmware embarcado: um buffer para armazenar bytes recebidos pela porta serial, uma tabela de valores de calibração de um sensor, ou um histórico circular das últimas N leituras de temperatura para calcular uma média

móvel. Como a memória RAM do ESP32 é limitada, o **tamanho** desses vetores costuma ser dimensionado com muito mais cuidado do que em um programa de computador — reservar um vetor de 10.000 posições “só por garantia” pode facilmente esgotar a memória disponível do chip.

CAPÍTULO 9

Ponteiros e Alocação Dinâmica de Memória

Ponteiros são, ao mesmo tempo, o recurso mais poderoso e o mais temido por quem está aprendendo C. Eles são também o motivo pelo qual C permanece tão relevante em programação de baixo nível: nenhuma outra construção da linguagem dá ao programador controle tão direto sobre a memória do computador.

9.1 O que é um ponteiro

Toda variável, ao ser declarada, ocupa um espaço na memória identificado por um **endereço**. Um **ponteiro** é uma variável cujo valor é o endereço de outra variável — em vez de armazenar um dado diretamente, ele aponta para onde esse dado está guardado.

Endereços e ponteiros

```
#include <stdio.h>

int main(void) {
    int idade = 25;
    int *ptr_idade = &idade; // ptr_idade guarda o ENDEREÇO de idade

    printf("Valor de idade:      %d\n", idade);
    printf("Endereco de idade:    %p\n", (void *) &idade);
    printf("Valor de ptr_idade:    %p\n", (void *) ptr_idade);
    printf("Valor apontado por ptr: %d\n", *ptr_idade);

    return 0;
}
```

Dois operadores são essenciais para trabalhar com ponteiros:

- **&** (*address-of*) — obtém o endereço de uma variável: `&idade`;
- ***** (*dereference*, ou “desreferenciação”) — acessa o valor armazenado no endereço apontado: `*ptr_idade`.

O mesmo símbolo ***** tem dois papéis diferentes dependendo do contexto: na **declaração** (`int *ptr`), ele indica que a variável é um ponteiro; em uma **expressão** (`*ptr = 10`), ele desreferencia o ponteiro, acessando o valor apontado.

9.2 Ponteiros como parâmetros de função

Como vimos no Capítulo 7, argumentos são passados por valor em C — uma função não pode alterar diretamente uma variável do código que a chamou. Ponteiros resolvem exatamente esse problema: em vez de passar o valor, passa-se o *endereço*, permitindo que a função modifique o dado original.

Modificando uma variável via ponteiro

```
#include <stdio.h>

void dobrar(int *x) {
    *x = *x * 2;    // modifica o valor no endereço apontado
}

int main(void) {
    int numero = 10;
    dobrar(&numero);
    printf("%d\n", numero);    // agora imprime 20
    return 0;
}
```

É exatamente por esse mecanismo que `scanf("%d", &idade)` funciona: sem o `&`, a função receberia apenas uma cópia do valor de `idade` e não teria como escrever o dado lido de volta na variável original.

9.3 Ponteiros e vetores

O nome de um vetor, usado sozinho em uma expressão, é convertido automaticamente para um ponteiro ao seu primeiro elemento — é por isso que, no Capítulo 8, passar um vetor para uma função na prática passa um ponteiro.

Aritmética de ponteiros

```
#include <stdio.h>

int main(void) {
    int valores[4] = {10, 20, 30, 40};
    int *p = valores;    // p aponta para valores[0]

    printf("%d\n", *p);    // 10
    printf("%d\n", *(p + 1));    // 20 (equivalente a valores[1])
    printf("%d\n", *(p + 2));    // 30 (equivalente a valores[2])

    return 0;
}
```

Somar 1 a um ponteiro não avança um único byte — avança o **tamanho do tipo apontado**. Em `int *p`, `p + 1` avança 4 bytes (o tamanho de um `int`), apontando corretamente para o próximo elemento do vetor.

9.4 Alocação dinâmica de memória

Todas as variáveis vistas até agora têm tamanho fixo, decidido em tempo de compilação, e residem na **pilha** (*stack*) — uma região de memória gerenciada automaticamente. Às vezes, porém, o tamanho necessário só é conhecido em tempo de execução (por exemplo, depende de uma entrada do usuário). Para esses casos, C permite reservar memória manualmente, em uma região chamada **heap**, usando funções da biblioteca `<stdlib.h>`.

Função	Finalidade
<code>malloc(tamanho)</code>	aloca <code>tamanho</code> bytes, sem inicializar
<code>calloc(n, tamanho)</code>	aloca <code>n</code> blocos de <code>tamanho</code> bytes, zerados
<code>realloc(ptr, novo_tam)</code>	redimensiona um bloco previamente alocado
<code>free(ptr)</code>	libera a memória apontada por <code>ptr</code>

Tabela 9.1: Funções de alocação dinâmica de `<stdlib.h>`.

Alocando um vetor em tempo de execução

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n;
    printf("Quantos numeros deseja armazenar? ");
    scanf("%d", &n);

    int *vetor = (int *) malloc(n * sizeof(int));
    if (vetor == NULL) {
        printf("Falha ao alocar memoria!\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        vetor[i] = i * i;
        printf("%d ", vetor[i]);
    }
    printf("\n");

    free(vetor); // devolve a memoria ao sistema
    return 0;
}
```

Toda chamada a `malloc` (ou `calloc/realloc`) bem-sucedida deve ter, mais cedo ou mais tarde, uma chamada correspondente a `free`. Esquecer de liberar memória alocada causa **vazamento de memória** (*memory leak*) — o programa consome cada vez mais memória ao longo do tempo, até eventualmente esgotá-la. Usar um ponteiro *depois* de liberá-lo (um *dangling pointer*) ou liberá-lo duas vezes são igualmente perigosos, e podem causar comportamento indefinido ou travamentos difíceis de depurar.

O ESP32 tem apenas algumas centenas de kB de RAM disponíveis, compartilhados entre o programa, a pilha e o heap — e, diferente de um computador, não existe memória virtual em disco para “salvar” o sistema quando a memória se esgota: se um `malloc` falhar por falta de espaço, ele retorna `NULL`, e um firmware mal escrito que não verifica isso pode travar ou reiniciar sozinho (um *crash*). Por essa razão, boa parte do código embarcado profissional evita alocação dinâmica sempre que possível, preferindo vetores de tamanho fixo, definidos em tempo de compilação — sacrificando um pouco de flexibilidade em troca de comportamento previsível, essencial em sistemas que muitas vezes precisam funcionar ininterruptamente por meses.

CAPÍTULO 10

Structs e Enumerações

Até aqui, cada variável guardava um único valor de um único tipo. Esta seção apresenta ferramentas para agrupar dados relacionados em uma única unidade — essencial para modelar informações do mundo real, como a leitura de um sensor (valor, unidade, instante da medição) ou a configuração de um periférico.

10.1 Registros e structs

Uma `struct` agrupa múltiplas variáveis, de tipos possivelmente diferentes, sob um único nome — cada uma acessível por um **campo** (*field*).

Definindo e usando uma struct

```
#include <stdio.h>

typedef struct {
    char nome[20];
    float temperatura;
    int umidade;
} Leitura;

int main(void) {
    Leitura sala = {"Sala de estar", 23.5f, 60};

    printf("%s: %.1f C, %d%% umidade\n",
        sala.nome, sala.temperatura, sala.umidade);

    sala.temperatura = 24.0f; // acessando e alterando um campo
    return 0;
}
```

O `typedef` antes de `struct` cria um **apelido de tipo**: em vez de escrever `struct Leitura sala`; toda vez, basta `Leitura sala`; Campos são acessados com o operador ponto (`.`).

10.1.1 Structs e ponteiros

Quando se tem um **ponteiro** para uma struct (comum ao passá-la para funções, evitando copiar todos os seus campos), o acesso aos campos usa o operador seta (`->`) em vez do ponto:

Acessando campos via ponteiro

```
#include <stdio.h>

typedef struct {
    float temperatura;
    int umidade;
} Leitura;

void exibir(Leitura *l) {
    printf("%.1f C, %d%%\n", l->temperatura, l->umidade);
    // equivalente a: (*l).temperatura, (*l).umidade
}

int main(void) {
    Leitura sala = {23.5f, 60};
    exibir(&sala);
    return 0;
}
```

10.2 Enumerações (enum)

Uma `enum` define um conjunto nomeado de constantes inteiras, tornando o código mais legível do que usar números “mágicos” soltos pelo texto.

Definindo uma enumeração

```
#include <stdio.h>

typedef enum {
    ESTADO_AGUARDANDO, // vale 0
    ESTADO_CONECTANDO, // vale 1
    ESTADO_ATIVO,      // vale 2
    ESTADO_ERRO        // vale 3
} EstadoSistema;

int main(void) {
    EstadoSistema estado = ESTADO_CONECTANDO;

    if (estado == ESTADO_CONECTANDO) {
        printf("Conectando...\n");
    }

    return 0;
}
```

Por padrão, o primeiro valor de uma `enum` vale 0, e cada valor seguinte é o anterior mais 1 — mas isso pode ser sobrescrito explicitamente (`ESTADO_ERRO = 100`, por exemplo). Combinada com um `switch` (Capítulo 5), uma `enum` é a base clássica para implementar máquinas de estado em C.

`structs` e `enums` são ferramentas centrais para organizar o estado de um projeto de IoT. Uma `struct` agrupa naturalmente tudo o que pertence a uma mesma leitura de sensor — valor, unidade, instante da medição —, em vez de espalhar essa informação em várias variáveis soltas; uma `enum` representa, de forma legível, os diferentes **modos de operação** de um dispositivo (por exemplo, `MODO_NORMAL`, `MODO_ALERTA`, `MODO_ECONOMIA`), testados com um `switch` a cada iteração do `loop()`. Usaremos exatamente essa combinação no projeto integrador do Capítulo 20.

CAPÍTULO 11

Modularização: Bibliotecas e Múltiplos Arquivos

Programas pequenos cabem confortavelmente em um único arquivo `.c`. Programas maiores — e todo firmware real se enquadra nessa categoria — precisam ser organizados em múltiplos arquivos, cada um responsável por uma parte coerente do sistema. Este capítulo fecha a Parte I mostrando como C organiza esse tipo de estrutura.

11.1 Bibliotecas de software

Uma **biblioteca** é um conjunto de funções, tipos e constantes já prontos, reutilizáveis por qualquer programa que a inclua. C distingue dois tipos principais:

- **Biblioteca padrão** — faz parte da linguagem, vem com o compilador. Exemplos já vistos: `<stdio.h>` (entrada/saída), `<stdlib.h>` (alocação de memória, conversões, `rand`), `<string.h>` (manipulação de strings), `<math.h>` (funções matemáticas);
- **Bibliotecas de terceiros** — escritas por outras pessoas ou equipes para propósitos específicos (por exemplo, comunicação com um sensor específico), instaladas separadamente.

Algumas funções úteis de `string.h` e `math.h`

```
#include <stdio.h>
#include <string.h>
#include <math.h>

int main(void) {
    char nome[20] = "ESP32";

    printf("Tamanho: %zu\n", strlen(nome));           // 5
    strcat(nome, "-DevKit");                          // concatena
    printf("Concatenado: %s\n", nome);

    printf("Raiz quadrada de 2: %f\n", sqrt(2));      // 1.414214
    printf("2 elevado a 10: %f\n", pow(2, 10));      // 1024.000000

    return 0;
}
```

Funções de `<math.h>` costumam exigir a ligação explícita da biblioteca matemática ao compilar no Linux: `gcc programa.c -o programa -lm`. Esse tipo de detalhe de configuração de build é, como veremos, um dos problemas que o PlatformIO resolve

automaticamente.

11.2 Dividindo um programa em múltiplos arquivos

Um projeto C real costuma ser dividido em pares de arquivos:

- Um **cabeçalho** (`.h`) — contém apenas **declarações**: protótipos de funções, definições de `struct/enum`, macros. É o que outros arquivos precisam *ver* para usar esse módulo;
- Uma **implementação** (`.c`) — contém as **definições** completas correspondentes.

`sensor.h`

```
#ifndef SENSOR_H
#define SENSOR_H

float converter_para_celsius(int leitura_bruta);

#endif
```

`sensor.c`

```
#include "sensor.h"

float converter_para_celsius(int leitura_bruta) {
    return (leitura_bruta / 4095.0f) * 3.3f * 100.0f;
}
```

`main.c`

```
#include <stdio.h>
#include "sensor.h" // aspas: cabeçalho local do projeto

int main(void) {
    float temp = converter_para_celsius(2048);
    printf("Temperatura: %.1f C\n", temp);
    return 0;
}
```

Compilando múltiplos arquivos

```
$ gcc main.c sensor.c -o programa
$ ./programa
Temperatura: 165.0 C
```

Repare em `#ifndef SENSOR_H / #define SENSOR_H / #endif` envolvendo todo o cabeçalho: essa é a **guarda de inclusão** (*include guard*), que evita erros de redefinição caso o mesmo cabeçalho seja incluído, direta ou indiretamente, mais de uma vez pelo mesmo arquivo. É uma convenção tão universal em C que sua ausência é considerada um descuido grave.

Note também a diferença entre `#include <arquivo.h>` (com sinais de menor/maior, para bibliotecas do sistema) e `#include "arquivo.h"` (com aspas, para arquivos do próprio projeto) — introduzida rapidamente no Capítulo 1 e agora com sentido completo.

A divisão em módulos `.h/.c` é o padrão universal de organização de qualquer projeto sério em PlatformIO — veremos sua estrutura de pastas específica (`src/`, `include/`, `lib/`) no Capítulo 15. É especialmente valiosa em projetos de IoT que crescem além de um único arquivo: separar, por exemplo, a leitura de sensores (`sensores.h/sensores.c`) da lógica de conectividade ou da lógica de controle mantém cada parte do firmware testável e legível de forma independente.

Parte II

Sistemas Embarcados com o ESP32 e o PlatformIO

CAPÍTULO 12

Introdução aos Sistemas Embarcados

Com os fundamentos da linguagem C estabelecidos, a Parte II desta apostila muda de ambiente: em vez de compilar e rodar programas em um computador comum, vamos escrever firmware para um microcontrolador real, o ESP32. Este capítulo estabelece o vocabulário e o contexto necessários antes de abirmos o PlatformIO pela primeira vez, no próximo capítulo.

12.1 O que é um sistema embarcado

Um **sistema embarcado** é um sistema computacional dedicado a uma função específica, geralmente integrado a um equipamento maior — diferente de um computador de propósito geral, que roda uma variedade praticamente ilimitada de programas. Uma máquina de lavar, um roteador Wi-Fi, o painel de instrumentos de um carro, uma pulseira de atividade física: todos são sistemas embarcados.

Característica	Computador comum	Sistema embarcado
Propósito	genérico	dedicado a uma função
Memória RAM	gigabytes	kilobytes a poucos megabytes
Armazenamento	discos de alta capacidade	memória flash, tipicamente MB
Sistema operacional	completo (Linux, Windows)	ausente, mínimo, ou um RTOS
Entrada/saída	teclado, mouse, tela	sensores, atuadores, LEDs
Consumo de energia	watts	miliwatts a microwatts

Tabela 12.1: Sistemas embarcados vs. computadores de propósito geral.

Essas restrições — pouca memória, sem sistema operacional completo, energia limitada — são exatamente o motivo pelo qual C continua sendo a linguagem dominante nesse domínio: ela permite controle fino sobre o que o processador de fato executa, sem a sobrecarga (*overhead*) de linguagens de mais alto nível.

12.2 O microcontrolador ESP32

O ESP32 é uma família de **microcontroladores** (*system-on-a-chip*, SoC) desenvolvida pela Espressif Systems, que integra em um único chip:

- Um processador **Xtensa LX6 (ou LX7, em variantes mais recentes)**, tipicamente dual-core, operando a até 240 MHz;
- **Wi-Fi** (802.11 b/g/n) e **Bluetooth** (clássico e BLE) integrados;

- Memória **SRAM** interna (algumas centenas de kB) para variáveis e pilha durante a execução;
- Memória **Flash** externa (tipicamente 4 MB ou mais) onde o firmware compilado é armazenado permanentemente;
- Dezenas de pinos **GPIO** de uso geral, além de periféricos dedicados: conversores analógico-digitais (ADC), saídas PWM, interfaces I²C, SPI, UART, e mais.

“Microcontrolador” e “microprocessador” não são sinônimos. Um **microprocessador** (como o de um computador) é apenas a unidade de processamento — precisa de chips externos para memória, entrada/saída, etc. Um **microcontrolador** já integra processador, memória e periféricos em um único chip, pronto para ser incorporado a um produto com o mínimo de componentes adicionais. É exatamente esse nível de integração que torna o ESP32 tão popular em projetos de eletrônica e produtos comerciais.

12.3 Como um firmware é diferente de um programa comum

Ao escrever `hello_world.c` no Capítulo 1, o programa era executado sob um sistema operacional (Linux, Windows ou macOS), que gerenciava memória, agendamento de processos e acesso a dispositivos por trás dos panos. Em um microcontrolador programado da forma mais simples, não existe esse intermediário: o código compilado é o **único** programa em execução, com acesso direto ao hardware.

Isso muda a estrutura típica de um programa de duas formas centrais:

1. Não existe uma função `main` que “termina” — como adiantado no Capítulo 1, o firmware gira indefinidamente em torno de um **laço principal**, reagindo a eventos e atualizando o estado do sistema enquanto houver energia;
2. O programador é responsável por configurar diretamente os periféricos que deseja usar — não existe um driver de teclado ou de tela pronto: existe, por exemplo, um pino GPIO específico que precisa ser configurado como saída antes de poder acender um LED.

Tecnicamente, o ESP32 *não* roda “sem sistema operacional” — por baixo do framework que usaremos nesta apostila, o **Arduino**, existe o **ESP-IDF**, o SDK oficial da Espressif, construído sobre o **FreeRTOS**, um sistema operacional de tempo real (RTOS) leve, projetado especificamente para microcontroladores. O framework Arduino é, na prática, uma camada de conveniência sobre essa base: ele simplifica drasticamente a forma de escrever o firmware — como veremos já no próximo capítulo — sem esconder o acesso direto ao hardware que torna C tão adequado para essa tarefa.

Antes de colocar a mão no código, o próximo capítulo faz uma pausa conceitual: de onde vêm os microcontroladores, e como sua unidade de processamento funciona por dentro.

Esse pano de fundo prepara o terreno para o **PlatformIO**, o ambiente que usaremos a partir do Capítulo 14 para escrever, compilar e carregar código C no ESP32.

CAPÍTULO 13

Microcontroladores: Histórico e Arquitetura

Antes de configurar o ambiente de desenvolvimento e escrever firmware de verdade, vale entender de onde vêm os microcontroladores e como sua unidade de processamento funciona por dentro. Esse pano de fundo explica *por que* o firmware que escreveremos ao longo desta apostila é estruturado do jeito que é — e, em particular, por que o ESP32 precisa ser protegido por um mecanismo chamado **Watchdog Timer**, apresentado ao final deste capítulo.

13.1 Um breve histórico

Desenvolver um novo circuito eletrônico dedicado a cada novo projeto consome muito tempo e recursos. Cada nova função exigiria, literalmente, uma nova rede de portas lógicas projetada do zero. Foi para resolver esse problema que o engenheiro **Federico Faggin**, então na Intel, ajudou a desenvolver um circuito capaz de executar um **conjunto de operações em uma sequência definida**, armazenada em uma memória — em vez de um circuito fixo, um circuito *programável*. Essa mudança de perspectiva é o que separa um circuito puramente digital (como os circuitos combinacionais e sequenciais estudados em eletrônica digital) de um **processador**: em vez de recabear o hardware para cada nova tarefa, basta escrever uma nova sequência de instruções.

O primeiro produto comercial desse tipo foi o chip **Intel 4004**, desenvolvido para equipar a calculadora *Busicom 141-PF*.

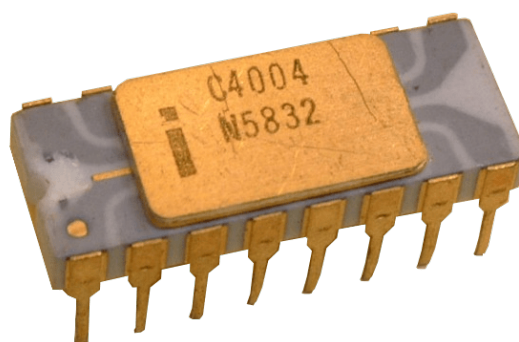


Figura 13.1: O Intel 4004 (1971), o primeiro microprocessador comercial.

Lançado em 1971, o Intel 4004 possuía uma arquitetura de apenas **4 bits**, um repertório de **45 instruções** diferentes, e operava a uma frequência de *clock* máxima de **740 kHz** — para comparação, milhares de vezes mais lento que o ESP32 que usaremos nesta apostila.

Um microprocessador sozinho, porém, não é suficiente para compor um circuito integrado completo para aplicações embarcadas: ele ainda precisaria de chips externos de memória e de interface para ser útil. Foi para resolver essa lacuna que surgiram os **microcontroladores** — circuitos integrados que reúnem, em um único substrato:

- a **ROM** (*Read-Only Memory*), associada às instruções do programa;
- a **RAM** (*Random-Access Memory*), associada aos dados manipulados durante a execução;
- registradores digitais de **interface** com dispositivos externos — os pinos GPIO e periféricos apresentados no Capítulo 12.

Esse é, essencialmente, o mesmo modelo — bastante amadurecido e miniaturizado — que encontramos hoje dentro do ESP32.

13.2 A unidade de processamento

A **unidade de processamento** (ou simplesmente *processador*) é o centro de um microcontrolador: é ela quem coordena, executa e gerencia todos os recursos disponíveis. Do ponto de vista da eletrônica digital, essa unidade é um **circuito sequencial** — ou seja, seu **estado atual** combinado aos **dados de entrada** determina qual será o **próximo estado** que o circuito assumirá.

Essas transições de estado acontecem de forma **síncrona**, ritmadas por um sinal elétrico periódico chamado **clock**. O intervalo de tempo entre duas transições consecutivas é chamado de **ciclo**.

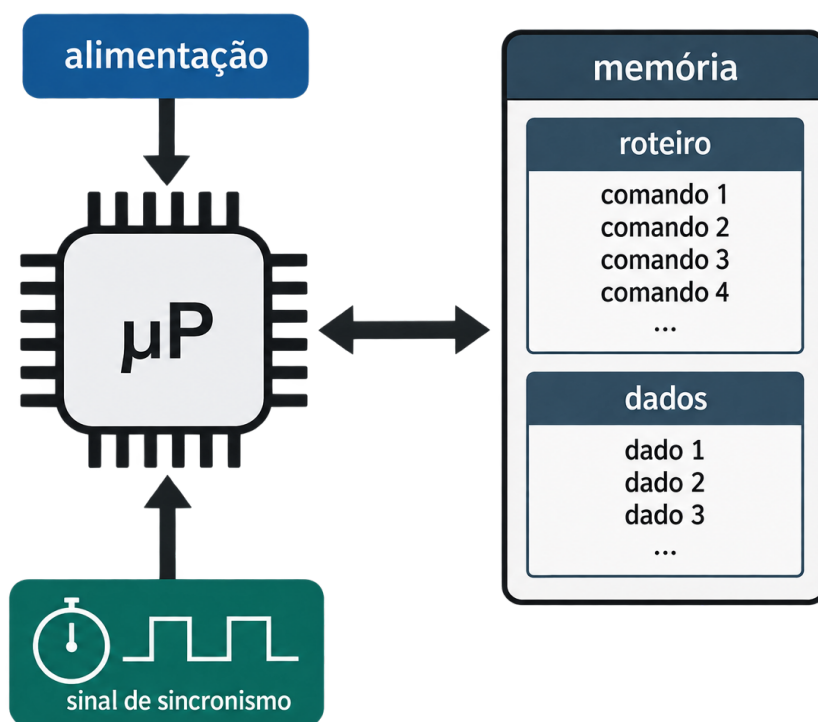


Figura 13.2: Visão simplificada de uma unidade de processamento (μ P): alimentação, sinal de sincronismo (*clock*) e o barramento bidirecional de acesso à memória, contendo o roteiro de instruções e os dados do programa.

Em cada ciclo, o processador busca a próxima instrução do **roteiro** (o programa, armazenado na memória de instruções), interpreta o que ela pede para fazer, e a executa — lendo ou escrevendo **dados**, atualizando registradores, ou desviando para outra instrução. É exatamente essa sequência de busca-decodificação-execução, repetida indefinidamente a cada ciclo de

clock, que dá vida ao código C que escrevemos: cada linha do seu programa, depois de compilada, se transforma em uma ou mais dessas instruções elementares.

Um aprofundamento completo sobre o projeto interno de processadores — unidades de controle, bancos de registradores, pipelines — foge do escopo desta apostila introdutória, mas é um tema fascinante para quem quiser entender ainda melhor o que acontece “por baixo” do código C.

13.3 Arquitetura de laço único (*Super Loop*)

Com o hardware entendido em linhas gerais, resta uma pergunta: como *organizar* o firmware que roda sobre ele? A resposta mais simples — e, ainda hoje, a mais usada em projetos de pequeno e médio porte — é a chamada **arquitetura de laço único** (em inglês, *Super Loop* ou *bare-metal loop*).

A ideia é direta: o firmware é dividido em duas fases.

1. Uma fase de **inicialização**, executada uma única vez, ao ligar o dispositivo — configurar pinos, iniciar periféricos, preparar variáveis;
2. Um **laço infinito**, executado indefinidamente depois disso, dentro do qual todas as tarefas do sistema são verificadas e atualizadas repetidamente, uma volta atrás da outra, para sempre.

Você já reconhece essa estrutura: é exatamente o par `setup()` / `loop()` do framework Arduino, apresentado no Capítulo 15. `setup()` é a fase de inicialização; `loop()`, chamada repetidamente pelo próprio framework, é o laço único.

A grande vantagem da arquitetura de laço único é a **simplicidade**: não há troca de contexto entre tarefas, não há concorrência para gerenciar, e o comportamento do sistema é fácil de prever e depurar — a cada volta do laço, o código roda sempre na mesma ordem. É também extremamente econômica em termos de memória, o que a torna ideal para microcontroladores simples.

O preço dessa simplicidade é que **nenhuma tarefa pode bloquear o laço por muito tempo**. Se uma única iteração de `loop()` demorar demais — por exemplo, presa em um `delay()` longo ou esperando um sensor lento —, *todo* o resto do sistema fica paralisado durante esse tempo: nenhum outro sensor é lido, nenhum outro botão é verificado. É exatamente por isso que o Capítulo 19 apresentou a técnica de temporização por `millis()` como alternativa ao `delay()` dentro de um laço com múltiplas responsabilidades — e é para proteger o sistema quando, apesar de tudo, alguma tarefa trava por completo, que existe o mecanismo apresentado a seguir.

13.4 Watchdog Timer

Mesmo em um firmware bem projetado, coisas dão errado: um sensor pode parar de responder e travar uma leitura que deveria ser rápida, um laço de espera pode nunca satisfazer sua condição de saída, ou um bug qualquer pode prender a execução em um ponto do código do qual ela nunca retorna. Em uma arquitetura de laço único (Seção 13.3), isso é particularmente grave: se `loop()` trava, **o sistema inteiro trava** — sem um sistema operacional completo para intervir, nada tira o processador desse estado sozinho.

A solução para esse problema é um pequeno circuito de hardware chamado **Watchdog Timer** (WDT, ou “temporizador de vigilância”): um contador regressivo independente que, se não for **reiniciado periodicamente pelo próprio firmware**, entende que o sistema travou e força um **reset** completo do microcontrolador — trazendo-o de volta a um estado conhecido e funcional, em vez de deixá-lo preso indefinidamente.

O uso típico segue um padrão de três passos:

1. **Configurar** o watchdog com um tempo limite (*timeout*) — por exemplo, 5 segundos;
2. Dentro do laço principal, **alimentar** (“dar comida a”) o watchdog periodicamente, sempre que o firmware sabe que está funcionando normalmente — isso reinicia a contagem regressiva;
3. Se, por qualquer motivo, o firmware travar e parar de alimentar o watchdog, o tempo limite se esgota e o **chip reinicia sozinho**.

src/main.cpp – protegendo o laço principal com o watchdog

```
#include <esp_task_wdt.h>

#define WDT_TIMEOUT 5 // segundos ate o reset, caso nao seja alimentado

void setup() {
    Serial.begin(115200);

    esp_task_wdt_init(WDT_TIMEOUT, true); // true: reinicia o chip ao vencer
    esp_task_wdt_add(NULL);              // registra a tarefa de loop() no
    ↪ watchdog

    Serial.println("Watchdog configurado. Sistema iniciado.");
}

void loop() {
    // ... leitura de sensores, logica do programa, etc ...

    Serial.println("Laco em execucao normal.");
    esp_task_wdt_reset(); // "alimenta" o watchdog: avisa que o laco esta vivo

    delay(1000);
}
```


Para ver o watchdog em ação, tente comentar a linha `esp_task_wdt_reset();` e substituir o `delay(1000)` por um laço vazio bem mais longo que o `WDT_TIMEOUT` configurado (por exemplo, `while (true) {}`). Poucos segundos depois, o watchdog vai perceber que o laço parou de ser alimentado e reiniciar o ESP32 sozinho — o monitor serial mostrará a mensagem "`Sistema iniciado.`" aparecendo repetidamente, uma prova visível de que o mecanismo de proteção está funcionando.

Chamar `esp_task_wdt_reset()` em um ponto do código onde, na verdade, algo já está errado — por exemplo, dentro de um laço de espera que deveria ter um limite de tentativas — **mascara o problema** em vez de resolvê-lo: o watchdog nunca vai disparar, mesmo que o sistema esteja preso ali, inútil, indefinidamente. O watchdog deve alimentar-se a partir de um ponto do código que só é alcançado quando o laço principal de fato completou uma volta inteira e saudável — nunca de dentro de uma espera que já é, ela própria, suspeita.

Com o histórico e a arquitetura de um microcontrolador em mente, o próximo capítulo coloca a mão na massa: a instalação e configuração do **PlatformIO**, o ambiente que usaremos para escrever, compilar e carregar firmware no ESP32.

CAPÍTULO 14

PlatformIO: Ambiente de Desenvolvimento

Para compilar e carregar código C no ESP32, precisamos de uma ferramenta capaz de: (1) compilar o código usando uma **toolchain cruzada** (o compilador roda no seu computador, mas gera código para o processador Xtensa do ESP32, como adiantado no Capítulo 2); (2) transferir o binário resultante para a memória flash do dispositivo via USB; e (3) abrir um canal de comunicação serial para depuração. O **PlatformIO** faz as três coisas — e muito mais — de forma integrada e gratuita.

14.1 O que é o PlatformIO

PlatformIO é um ecossistema de desenvolvimento profissional para sistemas embarcados, disponível como extensão para o Visual Studio Code (a forma mais comum de uso, e a que seguiremos aqui) ou como ferramenta de linha de comando independente. Ele suporta centenas de placas diferentes — não apenas o ESP32 — e múltiplos **frameworks** de desenvolvimento para cada uma.

Um **framework**, no contexto do PlatformIO, é o conjunto de bibliotecas e convenções sobre as quais o firmware é escrito. Para o ESP32, os dois frameworks mais usados são o **ESP-IDF** (*Espressif IoT Development Framework*, o SDK oficial da Espressif, escrito em C puro) e o **Arduino** — uma camada de conveniência sobre esse mesmo SDK, com uma API muito mais simples e enxuta, e o padrão de fato para projetos de prototipagem e IoT. É o framework **Arduino** que usaremos em todos os exemplos desta apostila daqui em diante.

Um detalhe técnico importante: arquivos do framework Arduino são compilados como **C++**, não C puro — é o que permite, por exemplo, que `Serial.print(...)` funcione (`Serial` é um objeto). Ainda assim, o estilo de código ensinado nesta apostila permanece deliberadamente próximo do C “puro” apresentado na Parte I — sem classes, templates ou os demais recursos exclusivos de C++ —, de modo que tudo o que você já aprendeu continua diretamente aplicável. Você vai reconhecer, na prática, que programar para o ESP32 com Arduino é escrever C dentro de um ambiente C++ minimalista.

14.2 Instalação

1. Instale o [Visual Studio Code](#) (disponível para Windows, macOS e Linux);
2. Dentro do VS Code, abra a aba de **Extensões** (ícone de blocos no menu lateral, ou `Ctrl+Shift+X`);
3. Busque por **“PlatformIO IDE”** e clique em **Instalar**;
4. Aguarde a instalação terminar — na primeira vez, o PlatformIO baixa e configura suas próprias dependências (Python, toolchains de compilação), o que pode levar alguns minutos;
5. Reinicie o VS Code quando solicitado. Um novo ícone de “formiga” (a logo do PlatformIO) deve aparecer na barra lateral esquerda.

Não é necessário instalar o Python ou qualquer toolchain manualmente — a extensão do PlatformIO cuida de todo esse processo de forma isolada do restante do sistema, evitando conflitos com outras instalações que você já possa ter.

14.3 Criando um novo projeto

Com a extensão instalada, clique no ícone da formiga na barra lateral para abrir o **PIO Home**, e em seguida em **New Project**. O assistente solicita quatro informações:

Campo	Valor sugerido
Name	<code>meu-primeiro-projeto</code> (ou o nome que preferir)
Board	<code>Espressif ESP32 Dev Module</code> (ajuste conforme sua placa)
Framework	<code>Arduino</code>
Location	a pasta onde o projeto será criado

Tabela 14.1: Configuração do assistente de novo projeto do PlatformIO.

O campo **Board** precisa corresponder (ou ao menos ser compatível com) a placa física que você possui. Existem dezenas de variantes de placas baseadas no ESP32 (DevKit V1, WROOM-32, NodeMCU-32S, entre outras) — quando em dúvida, `Espressif ESP32 Dev Module` é uma escolha segura e amplamente compatível para as placas de desenvolvimento mais comuns.

14.4 Anatomia de um projeto PlatformIO

Ao criar o projeto, o PlatformIO gera automaticamente a seguinte estrutura de pastas:

Estrutura de pastas de um projeto PlatformIO

```
meu-primeiro-projeto/
|-- platformio.ini      # arquivo de configuracao do projeto
|-- src/                # codigo-fonte principal do firmware
|   |-- main.cpp
|-- include/            # cabecalhos (.h) proprios do projeto
|-- lib/                # bibliotecas privadas do projeto
`-- test/               # testes automatizados (opcional)
```

- `src/` — onde vive o código-fonte principal. É aqui que criaremos e editaremos `main.cpp` em todos os exemplos seguintes (a extensão `.cpp`, e não `.c`, é exigida pelo framework Arduino, como visto na nota anterior);
- `include/` — para cabeçalhos (`.h`) específicos do seu projeto, como os vistos no Capítulo 11;
- `lib/` — para bibliotecas próprias, reutilizáveis entre projetos, ou de terceiros adicionadas manualmente;
- `platformio.ini` — o coração da configuração do projeto, detalhado a seguir.

14.4.1 O arquivo `platformio.ini`

platformio.ini

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
```

Chave	Significado
<code>platform</code>	a plataforma de hardware (<code>espressif32</code> , para toda a família ESP32)
<code>board</code>	o identificador exato da placa de desenvolvimento
<code>framework</code>	o SDK usado (<code>arduino</code> , nesta apostila)
<code>monitor_speed</code>	a velocidade (<i>baud rate</i>) do monitor serial, em bits por segundo

Tabela 14.2: Principais chaves de configuração do `platformio.ini`.

Esse arquivo texto simples — versionável junto ao código-fonte em um sistema como Git — é o que torna um projeto PlatformIO totalmente reproduzível em qualquer máquina: qualquer pessoa que clone o projeto e abra a pasta no VS Code com a extensão instalada terá exatamente o mesmo ambiente de compilação, sem precisar configurar nada manualmente.

14.5 A barra de tarefas do PlatformIO

Na parte inferior da janela do VS Code, a extensão adiciona uma barra de ícones com as ações mais usadas:

Ícone	Ação
✓ (visto)	Build — compila o projeto, sem carregá-lo na placa
→ (seta)	Upload — compila e carrega o firmware na placa via USB
⚡ (tomada)	Serial Monitor — abre o monitor serial para ver mensagens do firmware
🗑 (lixeira)	Clean — remove arquivos de compilação anteriores

Tabela 14.3: Principais ações da barra de tarefas do PlatformIO no VS Code.

O botão de **Upload** já executa o **Build** automaticamente antes de carregar — não é necessário clicar nos dois separadamente. Durante o upload, muitas placas exigem que se segure um botão físico chamado **BOOT** (ou **IO0**) no exato momento em que a transferência começa, para entrar em modo de gravação. Se o upload falhar com um erro de timeout de conexão, essa é a primeira coisa a verificar.

O próximo capítulo detalha o conteúdo mínimo de `src/main.cpp` em um projeto Arduino — o equivalente, no mundo embarcado, ao `hello_world.c` do Capítulo 1.

CAPÍTULO 15

Anatomia de um Projeto Arduino

Com o ambiente configurado, é hora de escrever o primeiro firmware de verdade. Este capítulo dissecta a estrutura mínima de um programa Arduino — o ponto de partida para todos os exemplos do restante da apostila.

15.1 As funções `setup` e `loop`

Diferente de um programa C convencional, onde tudo começa em `main`, um *sketch* Arduino (o nome tradicional dado a um programa nesse framework) é organizado em torno de duas funções obrigatórias:

src/main.cpp — esqueleto mínimo

```
void setup() {  
    // executado uma unica vez, ao ligar ou reiniciar o ESP32  
}  
  
void loop() {  
    // executado repetidamente, para sempre, enquanto houver energia  
}
```

- `setup()` roda **uma única vez**, logo após o ESP32 ser ligado ou reiniciado — é o lugar certo para inicializações: configurar pinos, iniciar a comunicação serial, preparar sensores;
- `loop()` roda **repetidamente, para sempre**, chamada automaticamente pelo próprio framework assim que `setup()` termina — é aqui que mora a lógica principal do firmware, exatamente o “laço infinito” adiantado no Capítulo 1 e no Capítulo 6.

Por que não uma função `main`? Tecnicamente, *existe* uma `main` nos bastidores — mas ela pertence ao próprio núcleo do framework Arduino, não ao seu código. Esse `main` interno cuida de inicializar o hardware e então chama `setup()` uma vez, seguida de `loop()` dentro de um laço infinito escondido de você. É por isso que `setup` e `loop` não retornam nenhum valor (`void`) e não recebem parâmetros: elas não são o ponto de entrada do programa, apenas dois pontos de extensão que o framework chama em momentos específicos.

15.2 Um primeiro exemplo completo

src/main.cpp – contador com atraso

```
void setup() {
    Serial.begin(115200);    // inicia a comunicacao serial
}

void loop() {
    static int contador = 0;

    Serial.print("Contador: ");
    Serial.println(contador);
    contador++;

    delay(1000);    // pausa de 1000 ms antes da proxima iteracao
}
```

A pausa acima usa `delay(ms)`, a função padrão do Arduino para aguardar um intervalo de tempo em milissegundos — **não** um laço vazio de espera ocupada (*busy-waiting*, como `for (long i = 0; i < 1000000; i++);`). Por baixo dos panos, o ESP32 roda sobre um sistema operacional de tempo real (o FreeRTOS, adiantado no Capítulo 12), que depende de que cada tarefa devolva o controle periodicamente; `delay()` já faz isso de forma segura, permitindo que outras tarefas do sistema (como a pilha de Wi-Fi) continuem funcionando durante a espera. Um laço vazio, em vez disso, prende o processador e pode disparar o **Watchdog Timer** — o mecanismo de segurança apresentado na Seção 13.4, que reinicia o chip sozinho quando percebe que o laço principal “travou”.

15.3 Compilando, carregando e monitorando

Com o código acima salvo em `src/main.cpp`, o fluxo de trabalho no PlatformIO é:

1. Conecte o ESP32 ao computador via cabo USB;
2. Clique no ícone de **Upload** (Capítulo 14) — o PlatformIO compila o projeto e carrega o binário na placa;
3. Clique no ícone de **Serial Monitor** para ver a saída de `Serial.print/Serial.println` em tempo real.

Saída esperada no monitor serial

```
Contador: 0
Contador: 1
Contador: 2
```


Se o monitor serial mostrar apenas caracteres ilegíveis (“lixo”), a causa quase sempre é uma incompatibilidade entre a velocidade configurada no monitor e a velocidade passada para `Serial.begin()` no código. Confirme que `monitor_speed` no `platformio.ini` coincide com o valor usado em `Serial.begin(...)` — nesta apostila, sempre `115200`.

15.4 De volta aos fundamentos: o que você já sabe aplicar

Vale parar um momento para notar que tudo neste primeiro exemplo já foi visto na Parte I: uma variável `static` preservando seu valor entre chamadas (Capítulo 7), incremento com `++` (Capítulo 3), e duas funções sem retorno (`void`, Capítulo 7). Programar o ESP32 não introduz uma linguagem nova — introduz um **framework** novo, com suas próprias funções e convenções, construído sobre exatamente o C que você já domina.

Os próximos capítulos exploram, um a um, os periféricos mais importantes do ESP32: entradas e saídas digitais (Capítulo 16), leitura de sensores analógicos (Capítulo 17), comunicação serial (Capítulo 18) e interrupções (Capítulo 19).

CAPÍTULO 16

Entradas e Saídas Digitais (GPIO)

GPIO (*General Purpose Input/Output*) são os pinos de propósito geral de um microcontrolador — a interface mais básica e mais usada entre o firmware e o mundo físico. Cada pino GPIO pode ser configurado por software como **saída** (o chip impõe um nível de tensão sobre ele — ligado ou desligado) ou como **entrada** (o chip lê o nível de tensão presente nele, imposto por algum circuito externo, como um botão).

O ESP32 tem várias dezenas de pinos GPIO, numerados (por exemplo, 2, 4, 5, ... 23). Nem todos podem ser usados livremente: alguns são reservados para a memória flash interna e não devem ser manipulados, outros têm comportamento especial ao ligar (*strapping pins*) e podem interferir no boot se conectados incorretamente. Consulte sempre o *datasheet* ou o diagrama de pinos (*pinout*) da sua placa específica antes de escolher quais pinos usar.

16.1 Configurando um pino como saída: o LED

O exemplo mais tradicional de programação embarcada — o equivalente ao *Hello, World!* — é piscar um LED, o chamado **blink**.

src/main.cpp — blink

```
#define PINO_LED 2

void setup() {
    pinMode(PINO_LED, OUTPUT);
}

void loop() {
    digitalWrite(PINO_LED, HIGH);    // liga o LED
    delay(500);

    digitalWrite(PINO_LED, LOW);     // desliga o LED
    delay(500);
}
```

Três funções do Arduino fazem todo o trabalho:

- `pinMode(pino, modo)` — define se o pino é **OUTPUT** (saída) ou **INPUT** (entrada), chamada uma única vez em `setup()`;

- `digitalWrite(pino, nivel)` — em modo saída, impõe o nível lógico `HIGH` (ligado, 3,3 V) ou `LOW` (desligado, 0 V);
- `delay(ms)` — pausa a execução pelo número de milissegundos indicado, como visto no Capítulo 15.

Os pinos GPIO do ESP32 operam em **3,3 V**, não em 5 V como muitas placas Arduino clássicas (como o Arduino Uno). Conectar um sinal de 5 V diretamente a um pino GPIO pode danificar permanentemente o chip. Sempre verifique o nível de tensão de qualquer módulo externo antes de conectá-lo.

16.2 Configurando um pino como entrada: o botão

Ler o estado de um botão segue a mesma lógica, com a direção invertida:

Lendo um botão com resistor de pull-up interno

```
#define PINO_LED      2
#define PINO_BOTAO   15

void setup() {
    pinMode(PINO_LED, OUTPUT);
    pinMode(PINO_BOTAO, INPUT_PULLUP); // resistor de pull-up interno
}

void loop() {
    bool pressionado = (digitalRead(PINO_BOTAO) == LOW);
    // com pull-up, o pino lê LOW quando o botao conecta o pino ao GND

    digitalWrite(PINO_LED, pressionado);
    delay(20); // pequena pausa (debounce simples)
}
```

Um botão simples, ligado entre um pino GPIO e o terra (GND), deixa o pino em um estado **flutuante** (indefinido) quando não está sendo pressionado, a menos que exista um resistor conectando-o a um nível de tensão conhecido. O modo `INPUT_PULLUP` ativa um resistor de *pull-up* interno ao próprio chip, eliminando a necessidade de um resistor externo para esse caso comum: o pino lê `HIGH` quando o botão está solto, e `LOW` quando pressionado (conectando o pino ao GND).

Botões mecânicos produzem múltiplas transições elétricas rápidas e espúrias no instante em que são pressionados ou soltos — um fenômeno chamado *bounce*. Um

pequeno atraso após cada leitura, como o `delay(20)` do exemplo, é uma forma simples (ainda que rudimentar) de filtrar esse ruído, chamada *debouncing*. O Capítulo 19 apresenta uma abordagem mais robusta, baseada em interrupções.

16.3 Controlando vários pinos com um vetor

Quando um projeto usa vários pinos com o mesmo papel — por exemplo, uma fileira de LEDs —, vale a pena aplicar o que vimos no Capítulo 8: guardar os números dos pinos em um **vetor** e percorrê-lo com um laço, em vez de repetir o mesmo código uma vez para cada pino.

Controlando vários LEDs com um vetor

```
const int pinos_leds[] = {2, 4, 5, 18};
const int quantidade = 4;

void setup() {
    for (int i = 0; i < quantidade; i++) {
        pinMode(pinos_leds[i], OUTPUT);
    }
}

void loop() {
    // acende um LED de cada vez, em sequencia (efeito "corrente")
    for (int i = 0; i < quantidade; i++) {
        digitalWrite(pinos_leds[i], HIGH);
        delay(150);
        digitalWrite(pinos_leds[i], LOW);
    }
}
```

Além de evitar repetição, essa abordagem torna o código mais fácil de estender: adicionar um quinto LED exige apenas um novo número no vetor `pinos_leds` (e ajustar `quantidade`), sem tocar no restante da lógica.

CAPÍTULO 17

Leitura de Sensores Analógicos (ADC)

Muitos sensores — potenciômetros, sensores de luminosidade (LDR), sensores de temperatura analógicos — não respondem apenas com “ligado” ou “desligado”: eles produzem uma **tensão contínua e variável**, proporcional à grandeza física medida. Para ler esse tipo de sinal, usamos um **ADC** (*Analog-to-Digital Converter*, ou conversor analógico-digital) — a porta de entrada de praticamente todo projeto de monitoramento IoT.

17.1 Como um ADC funciona

Um ADC amostra uma tensão analógica de entrada e a converte em um número inteiro, dentro de uma faixa determinada pela sua **resolução**, medida em bits. O ESP32 possui ADCs de **12 bits**, o que significa que cada leitura retorna um valor entre 0 e $2^{12} - 1 = 4095$ — quanto mais próxima a tensão de entrada estiver da tensão máxima suportada, mais próximo de 4095 será o valor lido.

Se você já programou com placas Arduino clássicas (como o Arduino Uno), tenha atenção: lá, `analogRead()` retorna valores de **10 bits** (0 a 1023). No ESP32, por padrão, a resolução é de **12 bits** (0 a 4095) — o dobro de precisão, mas também uma faixa de valores diferente da que você talvez esteja acostumado.

O ESP32 possui duas unidades ADC independentes: **ADC1** (8 canais, nos pinos GPIO32 a GPIO39) e **ADC2** (10 canais, compartilhados com o rádio Wi-Fi — por isso, leituras em ADC2 podem falhar ou ficar imprecisas enquanto o Wi-Fi está ativo). Por essa razão, esta apostila usa exclusivamente pinos da **ADC1** nos exemplos a seguir.

Pino GPIO	Unidade ADC
GPIO32	ADC1
GPIO33	ADC1
GPIO34	ADC1
GPIO35	ADC1
GPIO36	ADC1
GPIO39	ADC1

Tabela 17.1: Alguns pinos da ADC1 disponíveis no ESP32.

17.2 Lendo um sensor analógico

No framework Arduino, uma única função basta para realizar uma leitura: `analogRead(pino)`.

src/main.cpp – lendo um potenciômetro ou LDR

```
#define PINO_SENSOR 34

void setup() {
    Serial.begin(115200);
}

void loop() {
    int leitura_bruta = analogRead(PINO_SENSOR);
    float tensao = (leitura_bruta / 4095.0f) * 3.3f;

    Serial.print("Bruto: ");
    Serial.print(leitura_bruta);
    Serial.print(" | Aproximadamente: ");
    Serial.print(tensao);
    Serial.println(" V");

    delay(500);
}
```

A conversão $(leitura_bruta / 4095.0f) * 3.3f$ usada acima é uma **aproximação** — assume uma relação perfeitamente linear entre o valor lido e a tensão real, o que não é exatamente verdade na prática devido a não linearidades do hardware do ADC. Para a maioria dos sensores simples em um projeto de aprendizado — um LDR, um potenciômetro, um sensor de temperatura analógico — essa aproximação linear costuma ser suficiente.

17.3 De leitura bruta a informação: um sensor de luminosidade

Uma leitura analógica isolada raramente é o objetivo final — o valor bruto do ADC quase sempre precisa ser **traduzido** para uma grandeza com significado físico, e depois **registrado** ou **avaliado**. O exemplo a seguir lê um LDR (fotorresistor), estima um percentual de luminosidade e usa o próprio monitor serial como ferramenta de depuração e acompanhamento — a técnica de *print debugging* apresentada no Capítulo 4, agora aplicada a um sensor real:

Estimando luminosidade e registrando via Serial

```
#define PINO_LDR 34

void setup() {
```



```
    Serial.begin(115200);
}

void loop() {
    int leitura_bruta = analogRead(PINO_LDR);
    float luminosidade_pct = (leitura_bruta / 4095.0f) * 100.0f;

    Serial.print("[SENSOR] LDR bruto=");
    Serial.print(leitura_bruta);
    Serial.print(" luminosidade=");
    Serial.print(luminosidade_pct);
    Serial.println("%");

    if (luminosidade_pct < 20.0f) {
        Serial.println("[ALERTA] Ambiente escuro detectado!");
    }

    delay(1000);
}
```

17.4 Suavizando leituras com um vetor: média móvel

Sensores analógicos são naturalmente ruidosos — duas leituras consecutivas do mesmo ambiente raramente retornam exatamente o mesmo valor. Uma técnica simples e eficaz para suavizar esse ruído é a **média móvel**: manter um pequeno histórico das últimas N leituras (usando um vetor, como no Capítulo 8) e trabalhar sempre com a média desse histórico, em vez de uma leitura isolada.

Média móvel das últimas 10 leituras

```
#define PINO_SENSOR 34
#define TAMANHO_HISTORICO 10

int historico[TAMANHO_HISTORICO];
int indice_atual = 0;

void setup() {
    Serial.begin(115200);
    for (int i = 0; i < TAMANHO_HISTORICO; i++) {
        historico[i] = 0;
    }
}

void loop() {
    historico[indice_atual] = analogRead(PINO_SENSOR);
    indice_atual = (indice_atual + 1) % TAMANHO_HISTORICO;

    long soma = 0;
    for (int i = 0; i < TAMANHO_HISTORICO; i++) {
        soma += historico[i];
    }
}
```

```
}  
float media = (float) soma / TAMANHO_HISTORICO;  
  
Serial.print("[SENSOR] media movel = ");  
Serial.println(media);  
  
delay(100);  
}
```

O operador % (resto da divisão, Capítulo 3) faz o índice `indice_atual` voltar a 0 automaticamente ao chegar em `TAMANHO_HISTORICO` — o vetor se comporta como um **buffer circular**, sempre sobrescrevendo a leitura mais antiga com a mais nova. Essa é exatamente a técnica que usaremos no projeto integrador do Capítulo 20.

CAPÍTULO 18

Comunicação Serial e Depuração

Sem teclado, tela ou depurador visual conectado por padrão, a **comunicação serial** é a principal janela do desenvolvedor para dentro de um firmware em execução — é como saber o que o programa está “pensando” a cada momento, essencial tanto para aprender quanto para depurar problemas.

18.1 UART e o monitor serial

O ESP32 se comunica com o computador através de uma interface **UART** (*Universal Asynchronous Receiver/Transmitter*), tipicamente ligada a um chip conversor USB-serial na própria placa de desenvolvimento. No framework Arduino, essa comunicação é controlada pelo objeto `Serial`, que precisa ser iniciado uma única vez, em `setup()`, com a velocidade de comunicação desejada.

src/main.cpp – contador via Serial

```
void setup() {  
    Serial.begin(115200);  
}  
  
void loop() {  
    static int contador = 0;  
  
    Serial.print("Contador: ");  
    Serial.println(contador);  
    contador++;  
  
    delay(1000);  
}
```

Para visualizar essa saída, use o **Serial Monitor** do PlatformIO (Capítulo 14) — ele abre uma conexão serial com a placa na velocidade definida por `monitor_speed` no `platformio.ini` (que deve coincidir com o valor passado a `Serial.begin`) e exibe, em tempo real, tudo o que o firmware escreve.

Função	Comportamento
<code>Serial.print(x)</code>	imprime <code>x</code> , sem quebra de linha ao final
<code>Serial.println(x)</code>	imprime <code>x</code> , seguido de uma quebra de linha
<code>Serial.printf(fmt, ...)</code>	imprime com especificadores de formato, como <code>printf</code>

Tabela 18.1: Formas de escrever na porta serial no framework Arduino.

18.1.1 As variantes de `Serial.print`

`Serial.printf` aceita exatamente os mesmos especificadores de formato apresentados no Capítulo 4 (`%d`, `%f`, `%s`, `%.2f`, e assim por diante) — todo aquele conhecimento é diretamente reaproveitável aqui, bastando trocar `printf(...)` por `Serial.printf(...)`.

18.2 Saída como ferramenta de depuração, agora no firmware

A técnica de *print debugging* apresentada na Seção 4.6 — imprimir o valor de uma variável em pontos estratégicos do código para entender o comportamento de um programa — se transporta quase sem alterações para o firmware, usando `Serial` no lugar de `printf`:

Depurando uma leitura de sensor

```
int leitura_bruta = analogRead(PINO_SENSOR);
Serial.print("[DEBUG] leitura_bruta = ");
Serial.println(leitura_bruta);

float tensao = (leitura_bruta / 4095.0f) * 3.3f;
Serial.print("[DEBUG] tensao = ");
Serial.println(tensao);
```

18.3 Log por nível de severidade

O núcleo Arduino do ESP32 também oferece um sistema de **log por nível de severidade** — uma alternativa mais expressiva que `Serial.print` puro, com macros que já indicam a gravidade da mensagem:

Usando as macros de log

```
void setup() {
    Serial.begin(115200);
}

void loop() {
```

```

int leitura = analogRead(34);

log_i("Sistema em operacao normal.");
if (leitura < 100) {
    log_w("Leitura muito baixa: %d", leitura);
}
if (leitura == 0) {
    log_e("Sensor parece desconectado!");
}

delay(1000);
}

```

Macro	Nível	Uso típico
<code>log_e</code>	Error	falhas que impedem o funcionamento normal
<code>log_w</code>	Warning	situações anormais, mas não fatais
<code>log_i</code>	Info	mensagens informativas de operação normal
<code>log_d</code>	Debug	detalhes úteis apenas durante desenvolvimento
<code>log_v</code>	Verbose	o nível mais detalhado, quase sempre desativado

Tabela 18.2: Níveis de log disponíveis no núcleo Arduino do ESP32.

O nível mínimo de mensagens exibidas é controlado por uma opção de compilação, ajustável diretamente no `platformio.ini`:

platformio.ini com nível de log elevado

```

[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
build_flags = -DCORE_DEBUG_LEVEL=4 ; 0=Nenhum ... 5=Verbose

```

A grande vantagem de usar `log_x` em vez de `Serial.print` puro é justamente essa: o nível mínimo de log pode ser reduzido em uma versão “de produção” do firmware (silenciando mensagens de depuração detalhadas) sem precisar remover ou comentar cada chamada manualmente — basta alterar um único número em `platformio.ini`.

Uma armadilha clássica ao depurar por log em sistemas embarcados: mensagens de log *também* consomem tempo de processamento e, no caso da UART, banda de comunicação limitada. Excesso de logs dentro de um laço executado milhares de vezes

por segundo pode, ele próprio, atrasar o firmware o suficiente para mascarar ou até causar o problema que se está tentando diagnosticar — um efeito colateral que vale ter em mente ao decidir onde e com que frequência logar.

CAPÍTULO 19

Interrupções e Temporização

O botão lido no Capítulo 16 foi verificado por **sondagem** (*polling*): o laço principal pergunta repetidamente, a cada iteração, “o botão está pressionado agora?”. Essa abordagem é simples, mas tem um custo — o processador gasta tempo verificando um evento que, na maior parte do tempo, ainda não aconteceu. **Interrupções** resolvem esse problema de forma elegante: em vez de perguntar, o firmware é *avisado* pelo hardware exatamente no instante em que o evento ocorre.

19.1 O que é uma interrupção

Uma **interrupção** é um mecanismo de hardware que pausa imediatamente o que o processador está fazendo, executa uma função especial chamada **rotina de tratamento de interrupção** (*Interrupt Service Routine*, ou **ISR**), e depois retoma exatamente de onde havia parado — tudo de forma transparente ao código que estava em execução.

Uma ISR deve ser **extremamente curta e rápida**. Ela interrompe literalmente qualquer coisa que estivesse rodando, então mantê-la presa por muito tempo atrasa todo o resto do sistema. A prática recomendada — usada no exemplo a seguir — é: dentro da ISR, apenas **signalizar** que o evento ocorreu (atualizando uma variável) e sair; o processamento de fato acontece depois, dentro de `loop()`. Chamar `Serial.print`, `delay`, ou qualquer função “pesada” de dentro de uma ISR deve ser evitado.

19.2 Interrupções de GPIO com `attachInterrupt`

Vamos reescrever a leitura de botão do Capítulo 16, agora reagindo por interrupção em vez de sondagem.

src/main.cpp – botão via interrupção

```
#define PINO_BOTAO 15

volatile bool evento_botao = false;

void IRAM_ATTR botao_isr() {
    evento_botao = true;    // apenas sinaliza: nada de Serial aqui dentro
}
```

```
void setup() {
  Serial.begin(115200);
  pinMode(PINO_BOTAO, INPUT_PULLUP);

  // dispara a ISR na borda de descida (quando o botao e pressionado)
  attachInterrupt(digitalPinToInterrupt(PINO_BOTAO), botao_isr, FALLING);
}

void loop() {
  if (evento_botao) {
    evento_botao = false; // consome o evento
    Serial.println("Botao foi pressionado!");
  }
}
```

- `attachInterrupt(pino, funcao, modo)` registra `botao_isr` para ser chamada automaticamente pelo hardware sempre que o evento especificado ocorrer no pino indicado — `digitalPinToInterrupt()` converte o número do pino para o identificador de interrupção correspondente;
- `FALLING` dispara a interrupção na **borda de descida** — o instante exato em que o nível do pino cai de alto para baixo (o momento em que um botão com pull-up é pressionado). Outras opções incluem `RISING` (borda de subida) e `CHANGE` (qualquer mudança de nível);
- `IRAM_ATTR` garante que o código da ISR fique carregado na memória RAM interna do ESP32, exigido pelo framework para funções chamadas a partir de uma interrupção;
- A variável `evento_botao` é declarada `volatile` — exatamente pelo motivo explicado no Capítulo 7: seu valor é alterado de forma assíncrona pela ISR, e o compilador não pode assumir que ela permanece constante entre uma leitura e outra dentro de `loop()`.

Esse padrão — ISR mínima apenas sinalizando uma variável `volatile`, processamento de fato feito depois em `loop()` — também resolve o problema de *debounce* do Capítulo 16 de forma mais robusta: é possível ignorar novos eventos por um curto período após o último processado (comparando o tempo decorrido, como veremos a seguir), sem nunca bloquear a interrupção em si.

19.3 Temporização sem bloquear o laço: a técnica de `millis()`

Até aqui, todos os atrasos usaram `delay(ms)` — mas `delay` **bloqueia completamente** a execução de `loop()` durante aquele intervalo, impedindo que qualquer outra coisa aconteça nesse meio tempo (ler outro sensor, verificar outro botão, atualizar outro dispositivo). Para tarefas periódicas que não podem se dar ao luxo de bloquear o resto do programa, o Arduino oferece a função `millis()`, que retorna o número de milissegundos desde que o ESP32 foi ligado.

Piscando um LED sem usar `delay()`

```
#define PINO_LED 2
const unsigned long INTERVALO = 1000;    // 1000 ms

unsigned long ultimo_instante = 0;
bool estado_led = false;

void setup() {
    pinMode(PINO_LED, OUTPUT);
}

void loop() {
    unsigned long agora = millis();

    if (agora - ultimo_instante >= INTERVALO) {
        ultimo_instante = agora;
        estado_led = !estado_led;
        digitalWrite(PINO_LED, estado_led);
    }

    // o resto de loop() continua livre para fazer outras coisas
    // a cada volta, sem nunca ficar bloqueado esperando
}
```

Essa comparação — “já se passou tempo suficiente desde a última vez?” — é um padrão tão comum em firmware que tem nome: **temporização cooperativa** (ou, informalmente, o padrão *blink without delay*). Ao contrário de um `delay()`, `loop()` continua girando livremente a cada iteração; a ação só acontece quando o tempo decorrido atinge o intervalo desejado, deixando o restante do código livre para tratar sensores, botões e outras tarefas no meio tempo.

Combinar **interrupções** (para eventos externos imprevisíveis, como um botão) com **temporização por `millis()`** (para tarefas periódicas e previsíveis, como ler um sensor a cada segundo) é o padrão de projeto mais comum em firmware Arduino para o ESP32 — e é exatamente essa combinação que usaremos no projeto integrador do próximo capítulo.

CAPÍTULO 20

Projeto Integrador: Estação de Monitoramento IoT

Este capítulo final reúne, em um único projeto, praticamente tudo o que foi apresentado nesta apostila: tipos de dados, estruturas de controle, funções, vetores, `structs` e `enums`, além de GPIO, ADC, comunicação serial e interrupções. O objetivo é consolidar esses conceitos através de um firmware completo e funcional — não apenas um exemplo isolado de cada periférico.

20.1 Especificação do projeto

Uma **estação de monitoramento** de luminosidade ambiente: o ESP32 lê continuamente um sensor LDR, mantém uma média móvel das últimas leituras para suavizar ruído, registra o estado do ambiente pela porta serial e acende um LED de alerta sempre que o ambiente estiver escuro. Um botão alterna entre modo **normal** (só registra alertas) e modo **verboso** (registra cada leitura) — sem nunca bloquear o laço principal com `delay`.

20.1.1 Materiais

- 1 placa de desenvolvimento ESP32;
- 1 LED (com resistor limitador em série, $\sim 220\ \Omega$), como indicador de alerta;
- 1 sensor LDR, em um divisor de tensão com um resistor fixo ($\sim 10\ \text{k}\Omega$);
- 1 botão *push-button*;
- Protoboard e jumpers.

20.1.2 Ligações sugeridas

- LED de alerta → GPIO2 (através do resistor limitador, ao GND);
- Ponto médio do divisor de tensão do LDR → GPIO34 (ADC1);
- Botão → GPIO15 e GND (usando o *pull-up* interno, como no Capítulo 16).

20.2 Projetando a estrutura do estado

Antes de escrever o laço principal, vale modelar o **estado** do sistema com os tipos vistos no Capítulo 10 — uma prática de organização que se torna cada vez mais valiosa à medida que

um firmware cresce:

Modelando o estado do sistema

```
enum ModoOperacao {
    MODO_NORMAL,
    MODO_VERBOSO,
};

struct EstadoEstacao {
    ModoOperacao modo;
    int ultima_leitura;
    float media_movel;
    bool alerta_ativo;
};
```

Agrupar essas variáveis em uma única `struct`, em vez de espalhá-las como variáveis globais soltas, deixa explícito o que compõe “o estado do sistema” — e facilita, por exemplo, passar esse estado inteiro para uma função de log, caso o projeto cresça no futuro.

20.3 Firmware completo

src/main.cpp – estação de monitoramento IoT

```
#define PINO_LED      2
#define PINO_LDR      34
#define PINO_BOTAO    15

#define TAMANHO_HISTORICO  10
#define LIMiar_ESCURO      25.0f // abaixo disso, ambiente "escuro" (%)
#define INTERVALO_LEITURA 500UL // ms entre leituras do sensor

enum ModoOperacao {
    MODO_NORMAL,
    MODO_VERBOSO,
};

struct EstadoEstacao {
    ModoOperacao modo;
    int ultima_leitura;
    float media_movel;
    bool alerta_ativo;
};

EstadoEstacao estado = {MODO_NORMAL, 0, 0.0f, false};

int historico[TAMANHO_HISTORICO];
int indice_historico = 0;

volatile bool evento_botao = false;
unsigned long ultima_leitura_ms = 0;
```

```
void IRAM_ATTR botao_isr() {
    evento_botao = true;
}

float calcular_media(void) {
    long soma = 0;
    for (int i = 0; i < TAMANHO_HISTORICO; i++) {
        soma += historico[i];
    }
    return (float) soma / TAMANHO_HISTORICO;
}

void setup() {
    Serial.begin(115200);

    pinMode(PINO_LED, OUTPUT);
    pinMode(PINO_BOTAO, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(PINO_BOTAO), botao_isr, FALLING);

    for (int i = 0; i < TAMANHO_HISTORICO; i++) {
        historico[i] = 0;
    }

    Serial.println("Estacao de monitoramento iniciada em modo NORMAL.");
}

void loop() {
    // 1) verifica se o botao foi pressionado (sem bloquear o laço)
    if (evento_botao) {
        evento_botao = false;
        estado.moda = (estado.moda == MODO_NORMAL) ? MODO_VERBOSO : MODO_NORMAL;

        Serial.print("Modo alterado para: ");
        Serial.println(estado.moda == MODO_NORMAL ? "NORMAL" : "VERBOSO");
    }

    // 2) le o sensor periodicamente, sem usar delay()
    unsigned long agora = millis();
    if (agora - ultima_leitura_ms >= INTERVALO_LEITURA) {
        ultima_leitura_ms = agora;

        estado.ultima_leitura = analogRead(PINO_LDR);
        historico[indice_historico] = estado.ultima_leitura;
        indice_historico = (indice_historico + 1) % TAMANHO_HISTORICO;

        estado.media_movel = calcular_media();
        float luminosidade_pct = (estado.media_movel / 4095.0f) * 100.0f;
        estado.alerta_ativo = (luminosidade_pct < LIMIAR_ESCURO);

        digitalWrite(PINO_LED, estado.alerta_ativo);

        if (estado.moda == MODO_VERBOSO) {
```

```
        Serial.print("[LEITURA] bruta=");
        Serial.print(estado.ultima_leitura);
        Serial.print(" media=");
        Serial.print(estado.media_movel);
        Serial.print(" luminosidade=");
        Serial.print(luminosidade_pct);
        Serial.println("%");
    }

    if (estado.alerta_ativo) {
        Serial.println("[ALERTA] Ambiente escuro detectado!");
    }
}
```

20.4 Como o projeto se encaixa

- A `struct EstadoEstacao` e a `enum ModoOperacao` (Capítulo 10) organizam o estado do sistema de forma clara, em vez de espalhá-lo em variáveis globais soltas;
- O vetor `historico` (Capítulo 8) implementa um **buffer circular** para a média móvel, suavizando o ruído natural do sensor, exatamente como praticado no Capítulo 17;
- A leitura do LDR usa a **ADC** (Capítulo 17) para medir luminosidade de forma contínua, convertida de valor bruto para porcentagem com um `cast` (Capítulo 3);
- O botão é lido por **interrupção** (Capítulo 19), sinalizando o laço principal por meio de uma variável `volatile`, sem nunca bloquear a leitura contínua do sensor;
- A leitura periódica do sensor usa a técnica de **temporização por `millis()`** (Capítulo 19), mantendo `loop()` sempre livre para reagir ao botão a qualquer momento;
- Todo o comportamento é registrado pela **porta serial** (Capítulo 18) — mensagens de leitura apenas em modo verboso, e alertas sempre, independentemente do modo.

20.5 Para continuar praticando

Esta apostila termina aqui, mas o aprendizado de sistemas embarcados é, acima de tudo, prático. Como próximos passos a partir deste projeto, considere:

- Adicionar um segundo sensor (por exemplo, um sensor de temperatura) e expandir a `struct EstadoEstacao` para incluí-lo;
- Salvar o `modo` escolhido na memória flash (usando a biblioteca `Preferences` do Arduino para ESP32), para que a estação lembre a última configuração mesmo após ser desligada;
- Explorar a conectividade Wi-Fi do ESP32 para reportar as leituras a um servidor ou aplicativo remoto — o próximo passo natural de qualquer projeto de IoT;

- Reescrever o projeto dividindo-o em múltiplos arquivos `.h/.cpp` (Capítulo 11) — por exemplo, separando a lógica do sensor, do histórico e do botão em módulos independentes.

Cada um desses passos reutiliza e aprofunda os mesmos fundamentos de C apresentados nesta apostila — a linguagem muda pouco; o que cresce é a sua familiaridade com o hardware e o ecossistema ao redor dela.

CAPÍTULO A

Tabela ASCII

A tabela ASCII (*American Standard Code for Information Interchange*) define a correspondência entre valores numéricos inteiros e caracteres — é o que permite que um `char` em C seja, ao mesmo tempo, um número e um caractere. Os valores de 0 a 31 são **caracteres de controle**, não imprimíveis (como `\n` e `\t`, vistos no Capítulo 4).

Dec	Char	Dec	Char	Dec	Char	Dec	Char
32	(espaço)	56	8	80	P	104	h
33	!	57	9	81	Q	105	i
34	"	58	:	82	R	106	j
35	#	59	;	83	S	107	k
36	\$	60	<	84	T	108	l
37	%	61	=	85	U	109	m
38	&	62	>	86	V	110	n
39	'	63	?	87	W	111	o
40	(	64	@	88	X	112	p
41	)	65	A	89	Y	113	q
42	*	66	B	90	Z	114	r
43	+	67	C	91	[	115	s
44	,	68	D	92	\	116	t
45	-	69	E	93	]	117	u
46	.	70	F	94	^	118	v
47	/	71	G	95	_	119	w
48	0	72	H	96	'	120	x
49	1	73	I	97	a	121	y
50	2	74	J	98	b	122	z
51	3	75	K	99	c	123	{
52	4	76	L	100	d	124	
53	5	77	M	101	e	125	}
54	6	78	N	102	f	126	~
55	7	79	O	103	g		

Tabela A.1: Tabela ASCII imprimível (valores decimais 32 a 126).

Dec	Nome	Sequência de escape em C
0	NUL (nulo)	<code>\0</code>
7	BEL (alarme sonoro)	<code>\a</code>
8	BS (backspace)	<code>\b</code>
9	TAB (tabulação)	<code>\t</code>
10	LF (nova linha)	<code>\n</code>
13	CR (retorno de carro)	<code>\r</code>
27	ESC (escape)	—

Tabela A.2: Caracteres de controle mais relevantes para programação em C.

Como todo caractere é internamente um número, é possível fazer aritmética diretamente sobre `chars`. Por exemplo, `'a' - 'A'` vale 32 (a diferença de posição entre maiúsculas e minúsculas na tabela), e `('a' + 1)` vale `'b'` — uma técnica comum para converter maiúsculas em minúsculas ou percorrer o alfabeto em um laço.

CAPÍTULO B

Referência Rápida

Este apêndice reúne, em formato de consulta rápida, tabelas já apresentadas ao longo da apostila — útil como cola de referência durante os primeiros projetos, sem precisar percorrer os capítulos inteiros novamente.

B.1 Precedência de operadores

Da maior para a menor precedência (operadores na mesma linha têm precedência igual, avaliados conforme sua associatividade):

Operadores	Categoria
<code>() [] -> .</code>	chamada, indexação, acesso a membro
<code>! ~ ++ -- (tipo) * & sizeof</code>	unários, cast, ponteiro
<code>* / %</code>	multiplicativos
<code>+ -</code>	aditivos
<code><< >></code>	deslocamento de bits
<code>< <= > >=</code>	relacionais
<code>== !=</code>	igualdade
<code>&</code>	E bit a bit
<code>^</code>	OU exclusivo bit a bit
<code> </code>	OU bit a bit
<code>&&</code>	E lógico
<code> </code>	OU lógico
<code>?:</code>	ternário
<code>= += -= *= ...</code>	atribuição
<code>,</code>	vírgula

Tabela B.1: Precedência de operadores em C, do maior para o menor.

Na dúvida sobre a ordem de avaliação de uma expressão complexa, use parênteses explícitos — isso nunca está errado, e deixa a intenção do código clara tanto para o compilador quanto para quem lê depois.

B.2 Especificadores de formato (`printf/scanf`)

Especificador	Tipo
<code>%d / %i</code>	<code>int</code>
<code>%u</code>	<code>unsigned int</code>
<code>%ld / %lu</code>	<code>long / unsigned long</code>
<code>%f</code>	<code>float / double</code>
<code>%.Nf</code>	ponto flutuante com N casas decimais
<code>%c</code>	<code>char</code>
<code>%s</code>	string (<code>char *</code>)
<code>%x / %X</code>	hexadecimal (minúsculo / maiúsculo)
<code>%o</code>	octal
<code>%p</code>	endereço de ponteiro
<code>%zu</code>	<code>size_t</code> (retorno de <code>sizeof</code>)

Tabela B.2: Especificadores de formato mais usados.

B.3 Tipos de largura fixa (`stdint.h`)

Tipo	Intervalo
<code>uint8_t</code>	0 a 255
<code>int8_t</code>	−128 a 127
<code>uint16_t</code>	0 a 65 535
<code>int16_t</code>	−32 768 a 32 767
<code>uint32_t</code>	0 a $2^{32} - 1$
<code>int32_t</code>	− 2^{31} a $2^{31} - 1$

Tabela B.3: Tipos inteiros de largura fixa.

B.4 Funções essenciais do framework Arduino usadas nesta apostila

Função / macro	Finalidade
<code>pinMode(pino, modo)</code>	define um pino como <code>OUTPUT</code> , <code>INPUT</code> ou <code>INPUT_PULLUP</code>
<code>digitalWrite(pino, nivel)</code>	escreve nível lógico (<code>HIGH/LOW</code>) em um pino de saída
<code>digitalRead(pino)</code>	lê o nível lógico de um pino de entrada
<code>analogRead(pino)</code>	realiza uma leitura do ADC (0 a 4095 no ESP32)
<code>delay(ms)</code>	pausa cooperativa, em milissegundos
<code>millis()</code>	milissegundos desde a inicialização (para temporização sem bloqueio)
<code>Serial.begin(baud)</code>	inicia a comunicação serial na velocidade indicada
<code>Serial.print / Serial.println</code>	escreve na porta serial, com ou sem quebra de linha
<code>Serial.printf(fmt, ...)</code>	escrita formatada na porta serial (especificadores de <code>printf</code>)
<code>attachInterrupt(pino, isr, modo)</code>	associa uma função ISR a um pino (<code>RISING/FALLING/CHANGE</code>)
<code>digitalPinToInterrupt(pino)</code>	converte um número de pino no identificador de interrupção
<code>IRAM_ATTR</code>	garante que uma ISR fique carregada na RAM interna
<code>log_i / log_w / log_e</code>	mensagens de log por nível de severidade
<code>randomSeed / random</code>	inicializa a semente e gera números pseudoaleatórios

Tabela B.4: Funções do framework Arduino apresentadas na Parte II desta apostila.

